

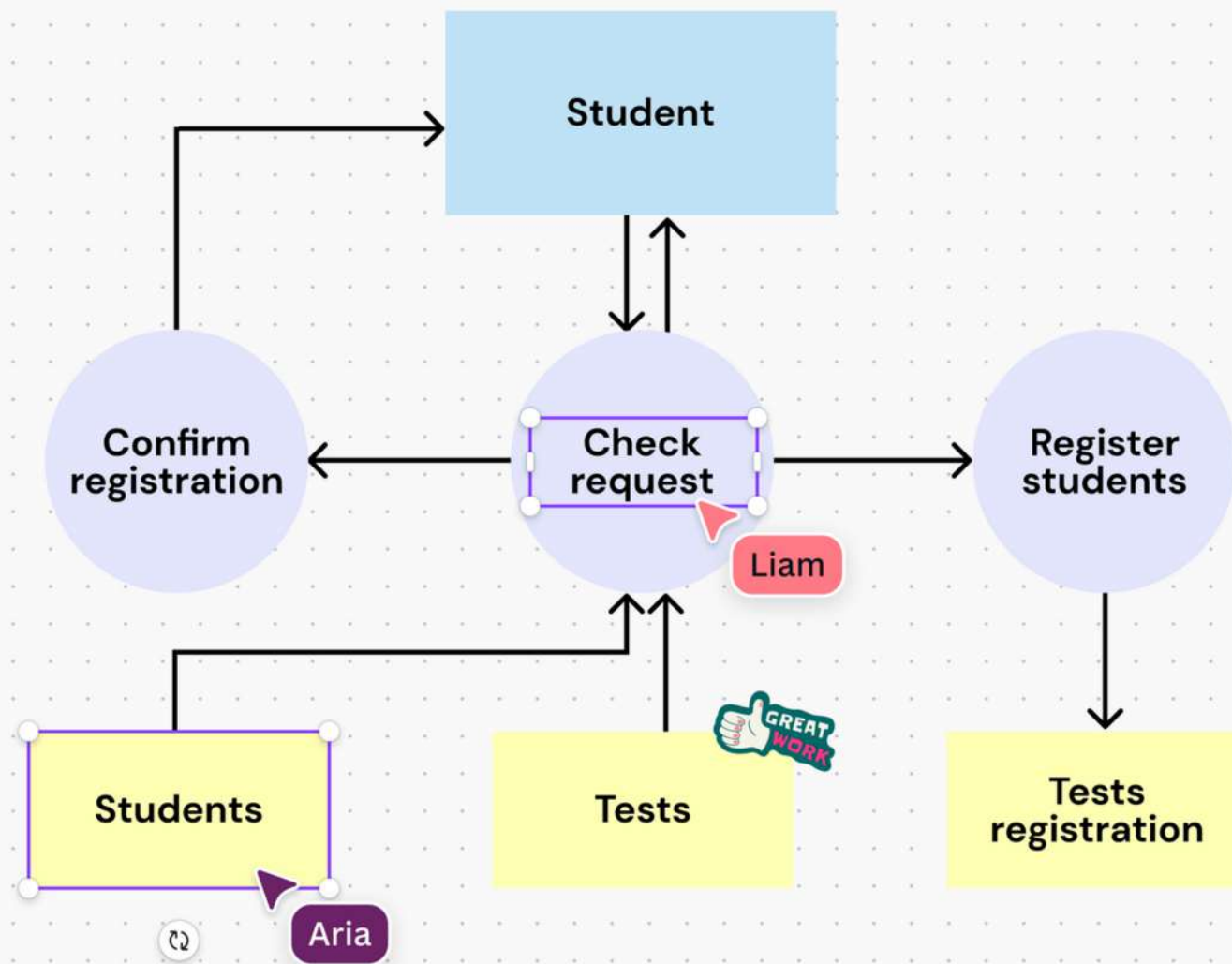
SYSTEM MODELLING

System Modelling Definition

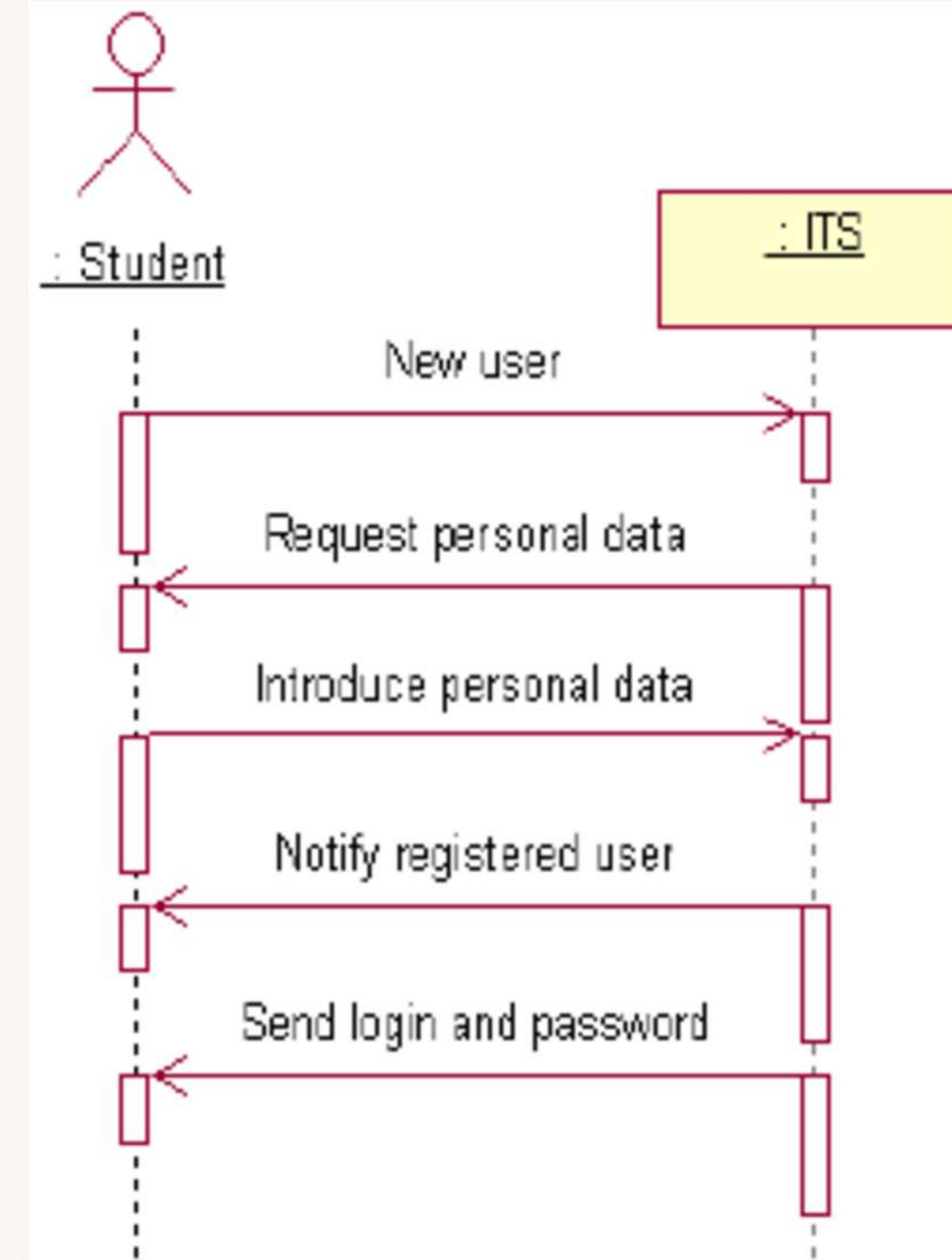


- **Definition:** It's the process of creating **abstract representations** of a system to understand, analyze, or communicate its structure and behavior.
- **Purpose:** Each model provides a **different viewpoint**—for example, how data flows, how components interact, or how users engage with the system.

(A system model is an abstraction of a system. Abstraction deliberately simplifies and leaves out details)

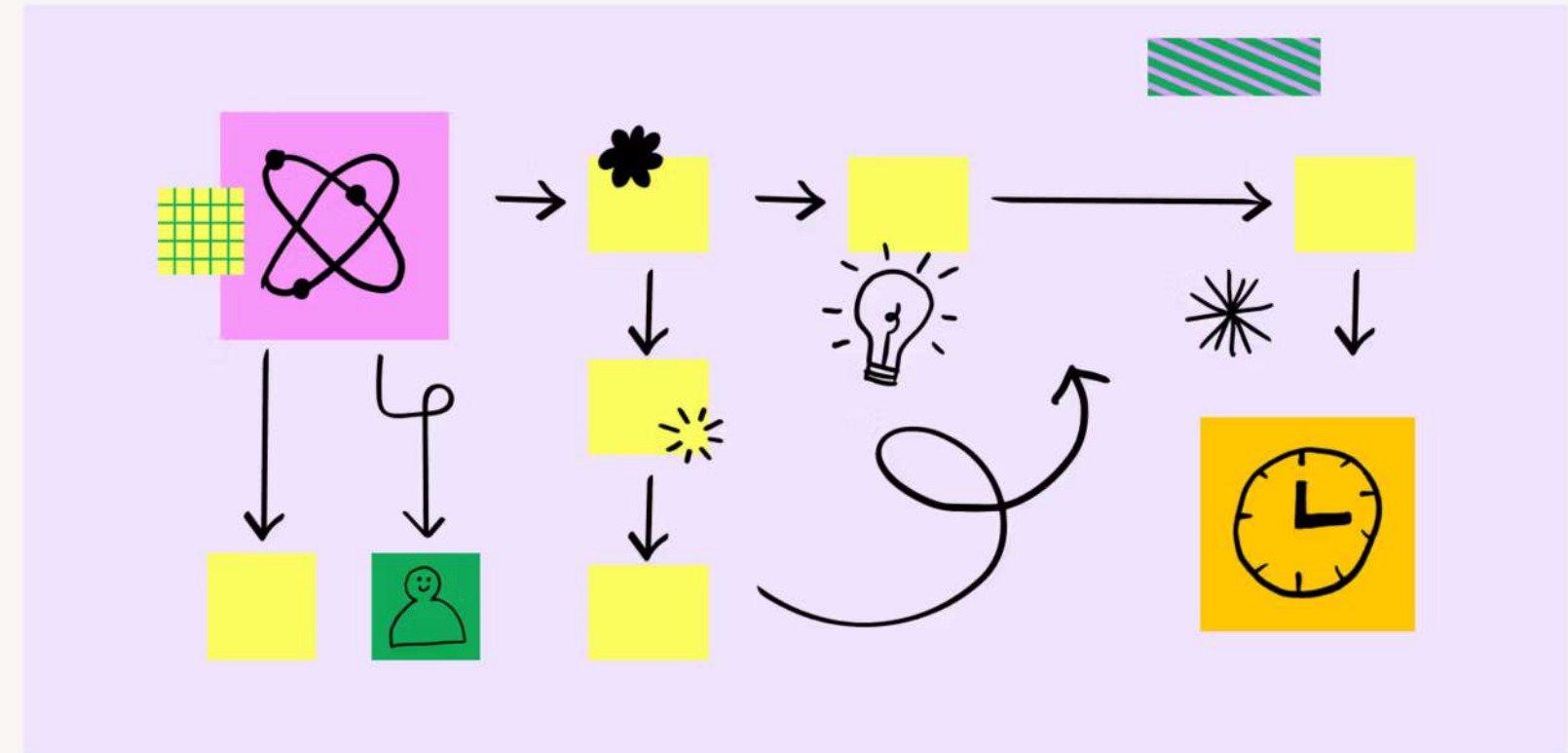


Data Flow Diagram



Sequence Diagram

Why It Matters



Communication

Different stakeholders (developers, managers, users) need different perspectives.

Validation

Models help check requirements before coding begins.

Documentation

They serve as a long-term reference for system design.

Models in Requirements Engineering

Models are used during the requirements engineering process to **help derive the detailed requirements for a system**, during the design process to describe the system to engineers implementing the system, and **after implementation to document the system's structure and operation**.

Models of the existing system

Models of the existing system are used during requirements engineering. They help clarify what the existing system does, and they can be used to focus a stake holder discussion on its strengths and weaknesses

Models of the new system

Models of the new system are used during requirements engineering to help explain the proposed requirements to other system stakeholders. Engineers use these models to discuss design proposals and to document the system for implementation.



Perspectives in System Modeling

It is important to understand that a system model is not a complete representation of system. It purposely leaves out detail to make it easier to understand. A model is an abstraction of the system being studied rather than an alternative representation of that system. A representation of a system should maintain all the information about the entity being represented.

Perspective	Purpose	Example	Typical UML Diagram
External Perspective	Shows the system in its environment and how it fits into a larger context	A library management system modeled as part of a university IT ecosystem, showing connections to student databases and payment gateways	Context Diagram or Use Case Diagram
Interaction Perspective	Focuses on how the system interacts with users, other systems, or internal components	Online shopping: customer places an order → system checks inventory → payment service processes transaction → shipping service updates status	Sequence Diagram or Communication Diagram
Structural Perspective	Describes the static organization of the system, including components and data structures	A hospital system with classes like <i>Patient</i> , <i>Doctor</i> , <i>Appointment</i> , and <i>MedicalRecord</i> , showing relationships	Class Diagram or Component Diagram
Behavioral Perspective	Models dynamic behavior, events, and state changes over time	ATM system: states like <i>Idle</i> → <i>Card Inserted</i> → <i>PIN Entered</i> → <i>Transaction Processing</i> → <i>Card Ejected</i>	State Machine Diagram or Activity Diagram

UML Modelling

Activity diagrams - the activities involved in a process or in data processing.

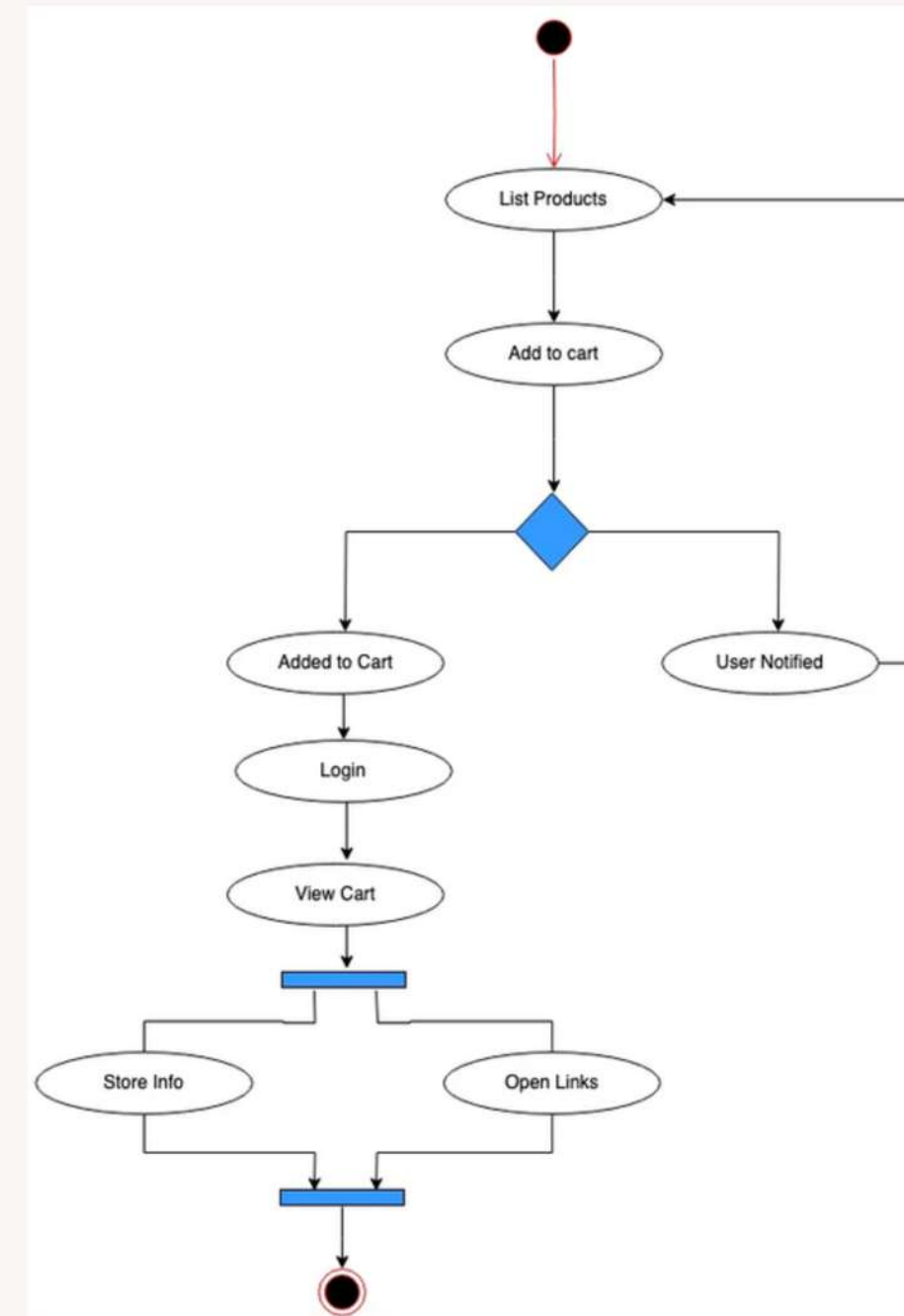
Use case diagrams - the functions exposed by the system and their connections to the actors in its context.

Sequence diagrams - interactions between actors and the system and between system components.

Class diagrams - object classes in the system and their relationships.

State machine diagrams - system behavior triggered by internal and external events.

UNIFIED MODELING LANGUAGE



UML Modelling

Activity diagrams - the activities involved in a process or in data processing.

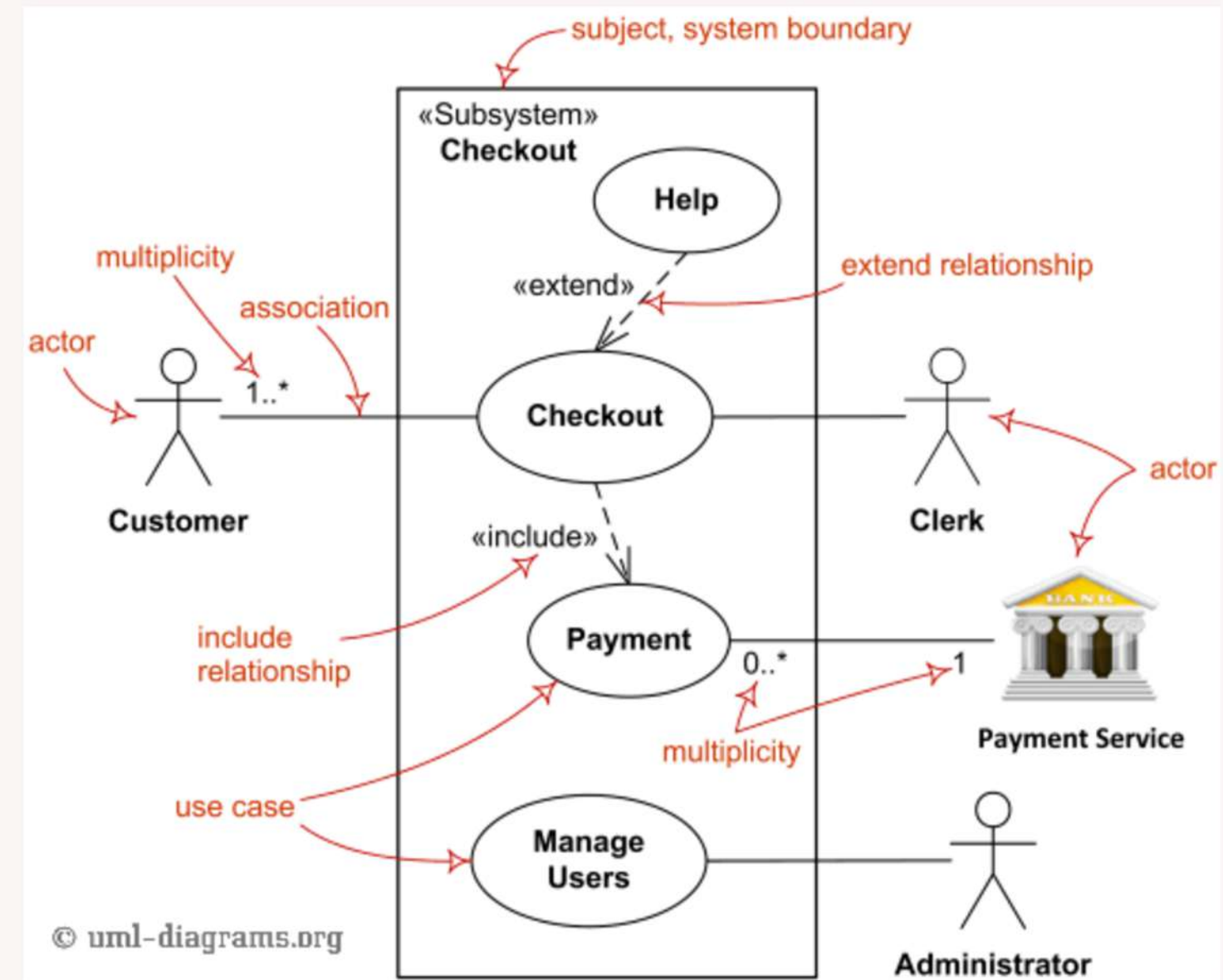
Use case diagrams - the functions exposed by the system and their connections to the actors in its context.

Sequence diagrams - interactions between actors and the system and between system components.

Class diagrams - object classes in the system and their relationships.

State machine diagrams - system behavior triggered by internal and external events.

UNIFIED MODELING LANGUAGE



UML Modelling

Activity diagrams - the activities involved in a process or in data processing.

Use case diagrams - the functions exposed by the system and their connections to the actors in its context.

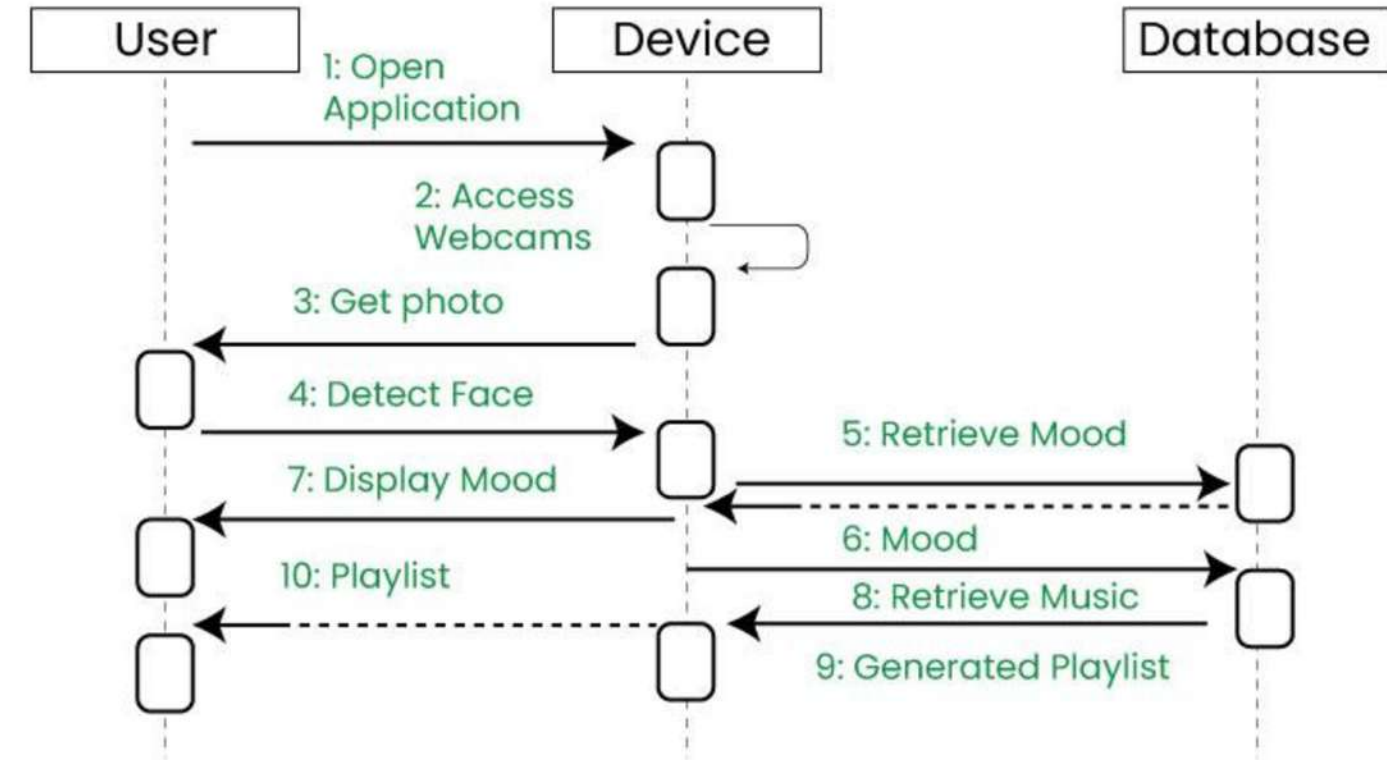
Sequence diagrams - interactions between actors and the system and between system components.

Class diagrams - object classes in the system and their relationships.

State machine diagrams - system behavior triggered by internal and external events.

UNIFIED MODELING LANGUAGE

Example sequence diagram



UML Modelling

Activity diagrams - the activities involved in a process or in data processing.

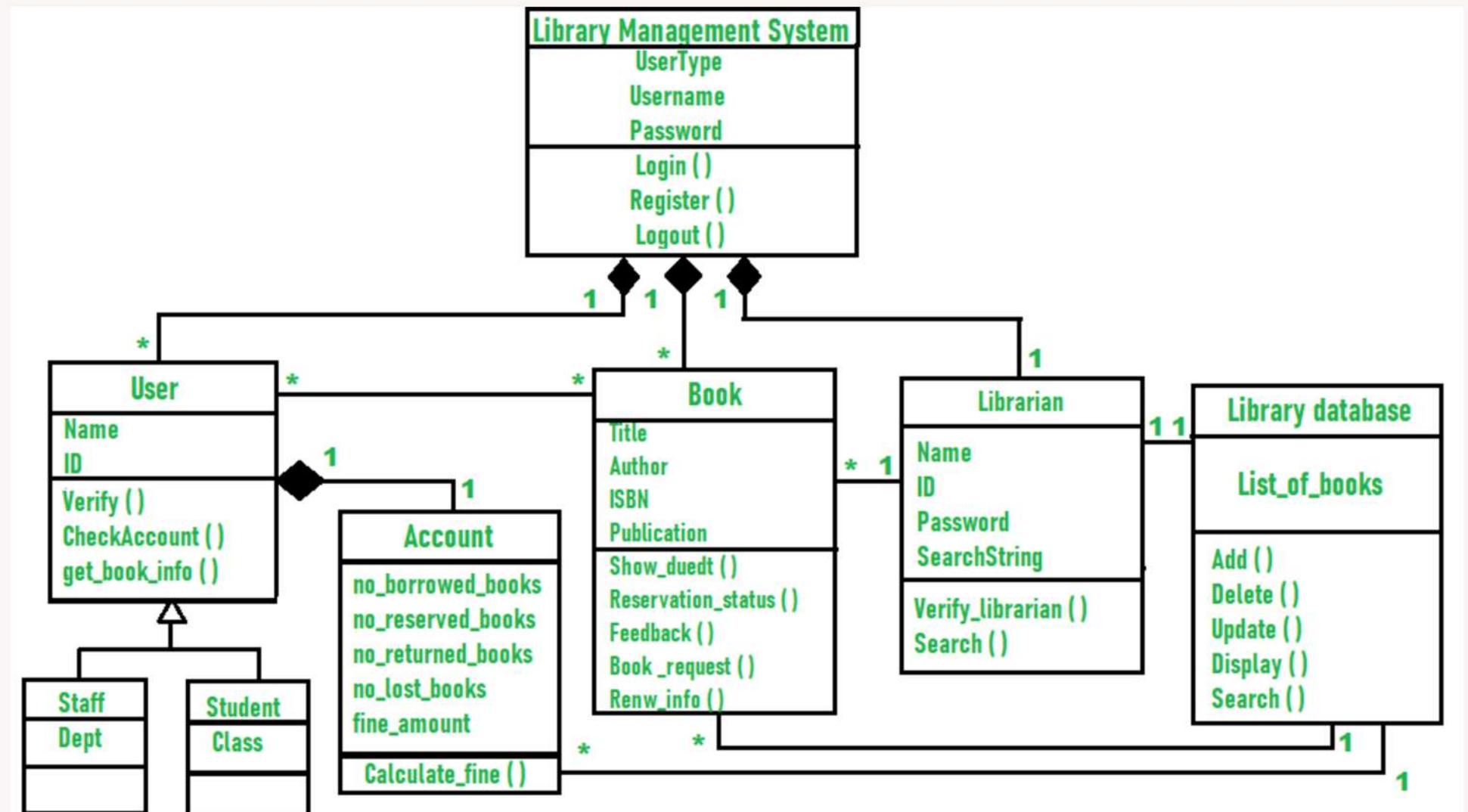
Use case diagrams - the functions exposed by the system and their connections to the actors in its context.

Sequence diagrams - interactions between actors and the system and between system components.

Class diagrams - object classes in the system and their relationships.

State machine diagrams - system behavior triggered by internal and external events.

UNIFIED MODELING LANGUAGE



CLASS DIAGRAM FOR LIBRARY MANAGEMENT SYSTEM

UML Modelling

Activity diagrams - the activities involved in a process or in data processing.

Use case diagrams - the functions exposed by the system and their connections to the actors in its context.

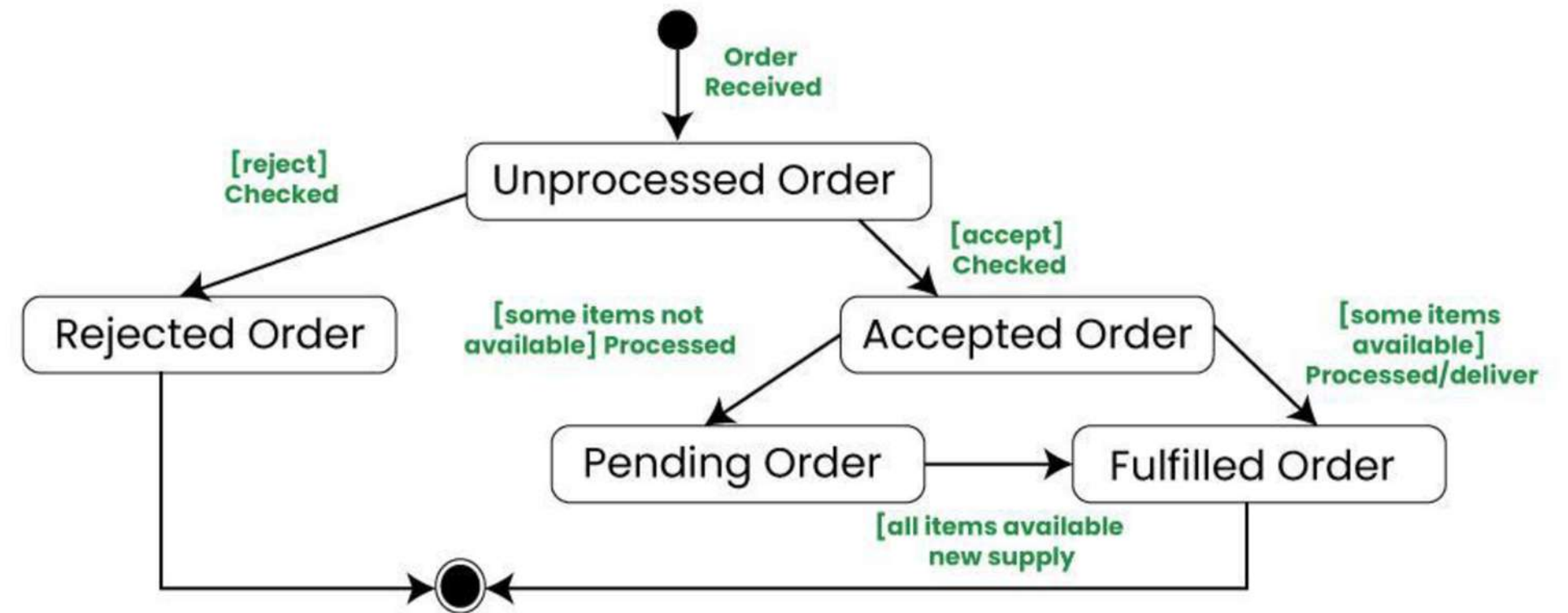
Sequence diagrams - interactions between actors and the system and between system components.

Class diagrams - object classes in the system and their relationships.

State machine diagrams - system behavior triggered by internal and external events.

UNIFIED MODELING LANGUAGE

State machine diagram for an online order



SYSTEM MODELLING

Context models

Interaction models

Structural models

Behavioral models

Data models

UI prototype

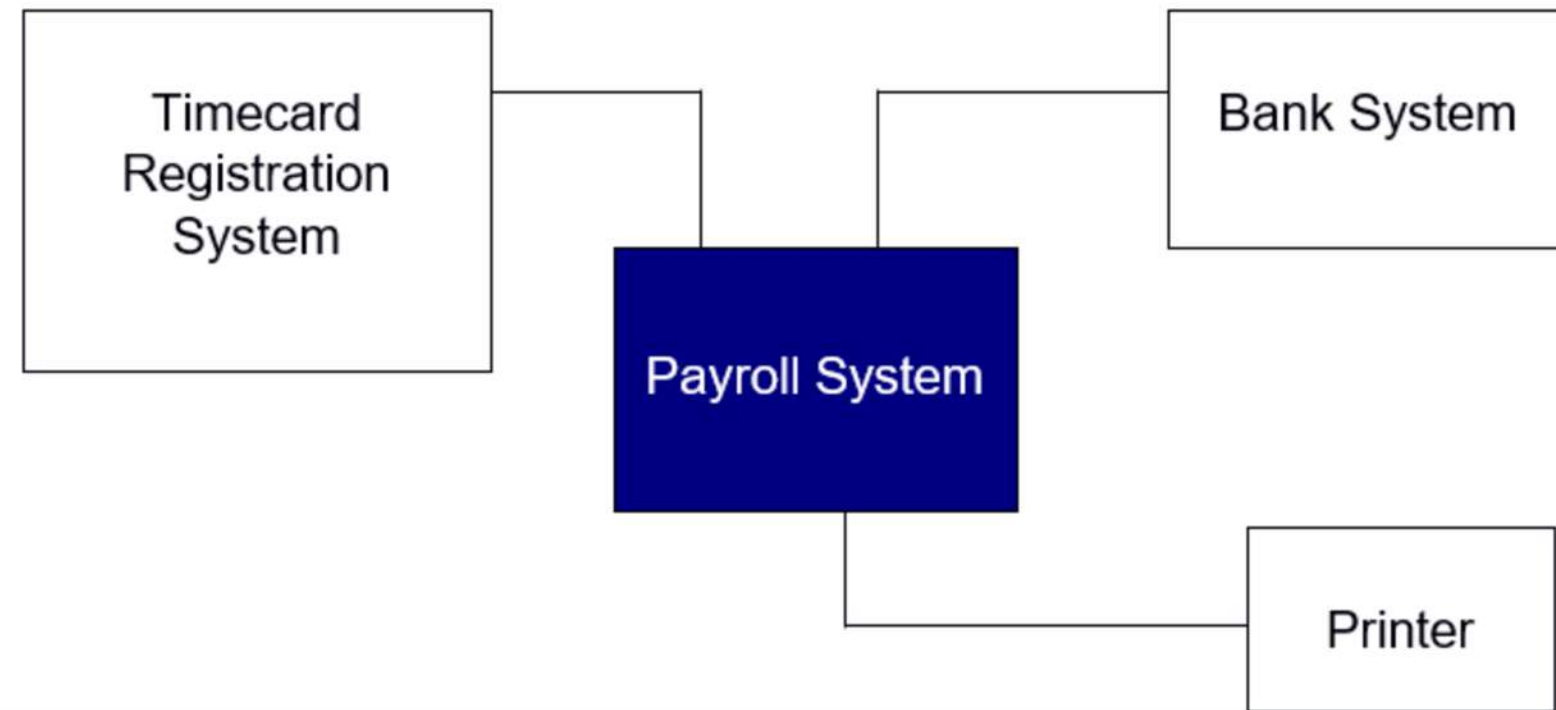
Model-driven engineering

UNIT II Chapter I

Context Models

At the early stage of system specification, defining system boundaries is one of the most critical tasks. It sets the scope of what the system will do and what lies outside its responsibility.

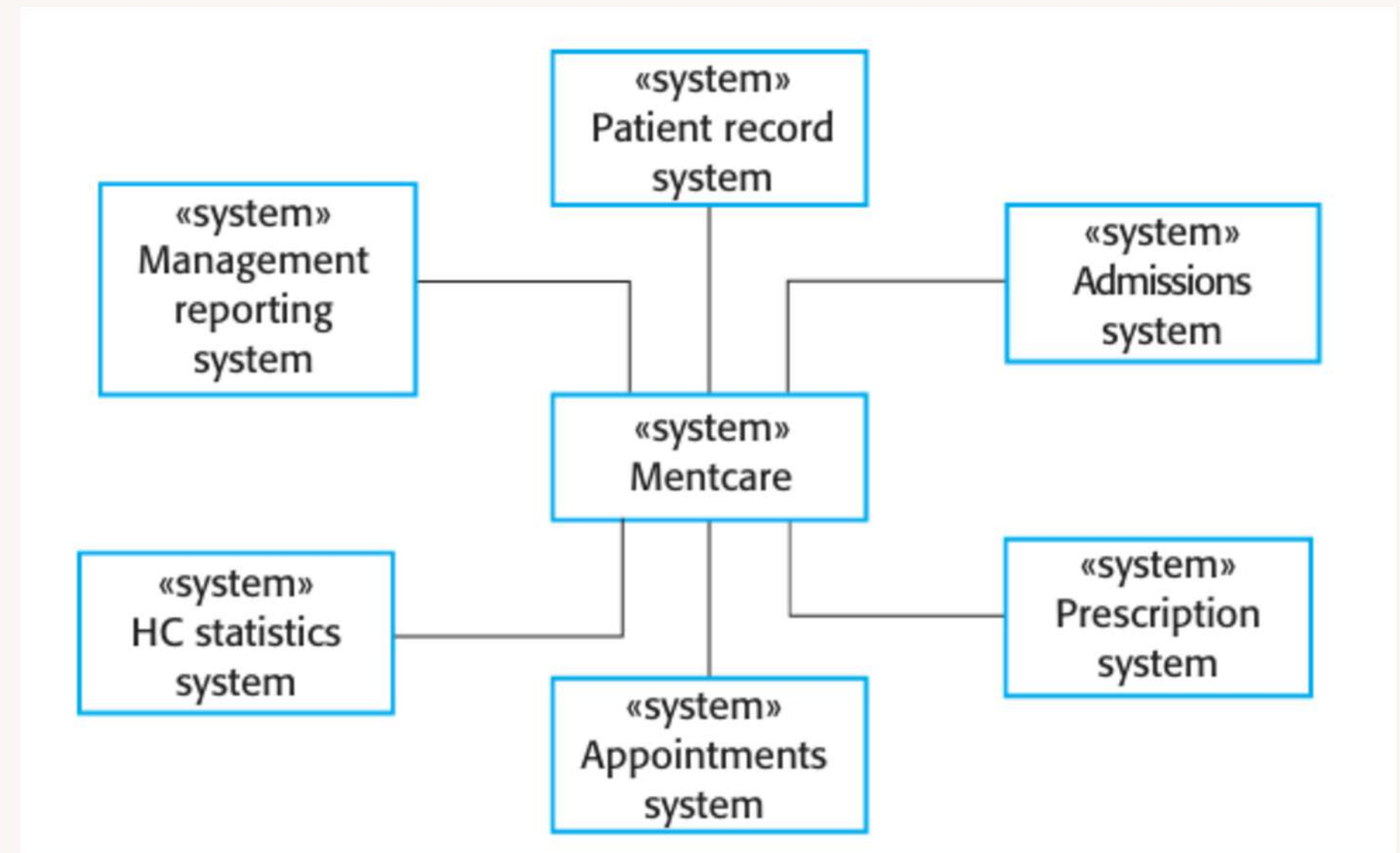
Image: High-level architectural models show the system and its relationship with other systems.



Why System Boundaries Matter

- **Clarity of Scope:** Prevents scope creep by explicitly stating what is inside vs. outside the system.
- **Cost Control:** Reduces unnecessary development by avoiding duplication of existing functionality.
- **Stakeholder Alignment:** Ensures all parties agree on expectations before design begins.
- **Integration Planning:** Helps identify overlaps with existing systems and where interfaces are needed.

Context Models



Context models simply show the other systems in the environment, not how the system being developed is used in that environment.

- Once boundaries are set, analysts define the system's context—its dependencies and environment.
- A context model (like the Mentcare example) shows:
 - Which external systems it connects to (appointments, patient records, reporting, admissions, prescriptions).
 - How data flows between them.
 - These models don't show relationship types in detail (e.g., whether data is shared, consumed, or produced), but they establish the big picture.

Business Process Models

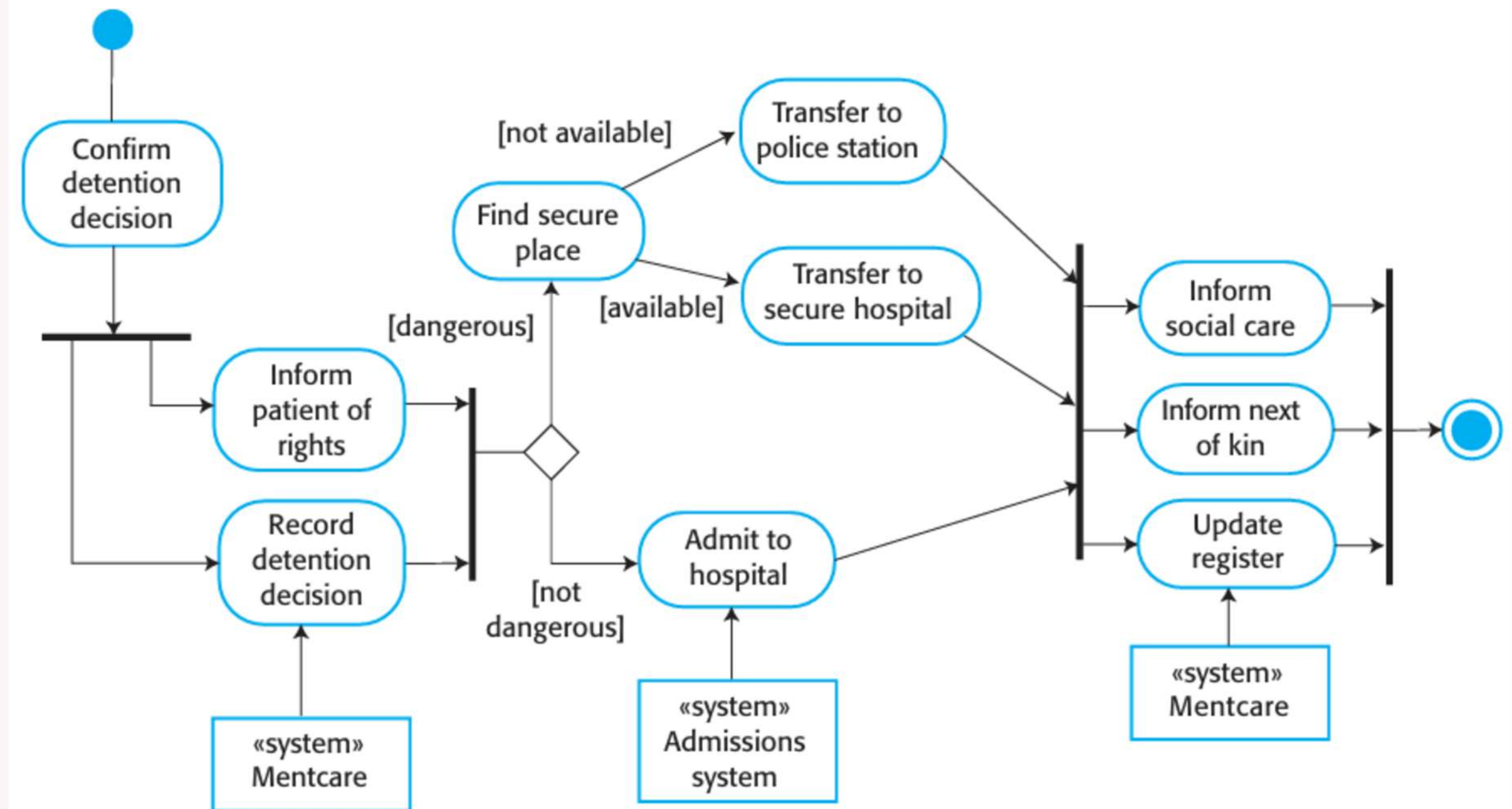
To complement context models, analysts use process models .

Together, context and process models help ensure:

- The system integrates smoothly with its environment.
- Legal, social, and organizational requirements are respected.
- Stakeholders understand both the scope and the role of the system.

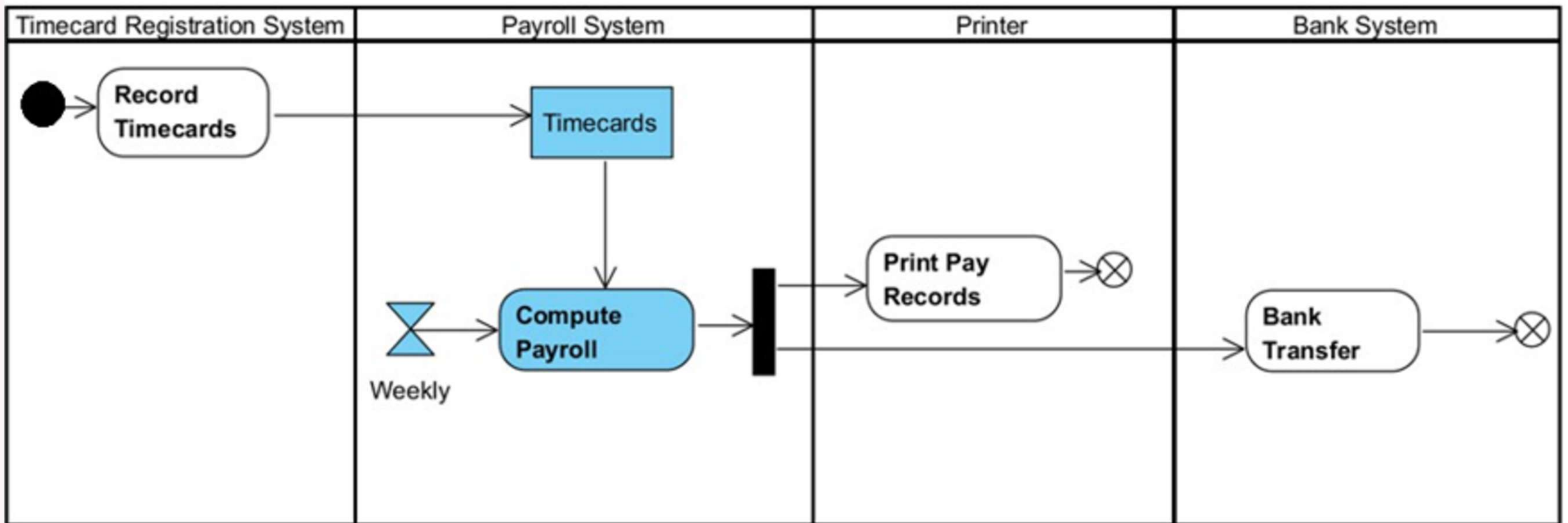
These show:

- Human and automated subprocesses.
- Flow of control, concurrency, and decision points.
- Where the system plays a role in critical processes (e.g., Mentcare ensuring legal safeguards in involuntary detention).



Process perspective

Example: Payroll system in its operational context



SYSTEM MODELLING

Context models

Interaction models

Structural models

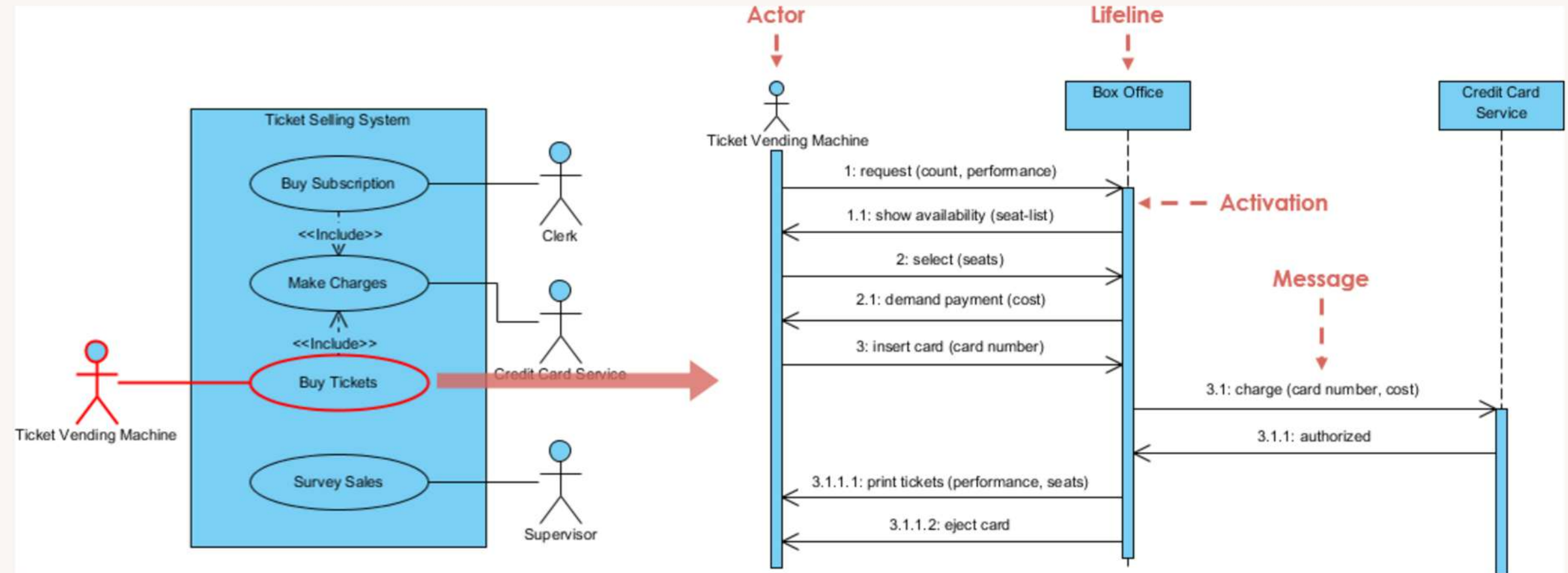
Behavioral models

Data models

UI prototype

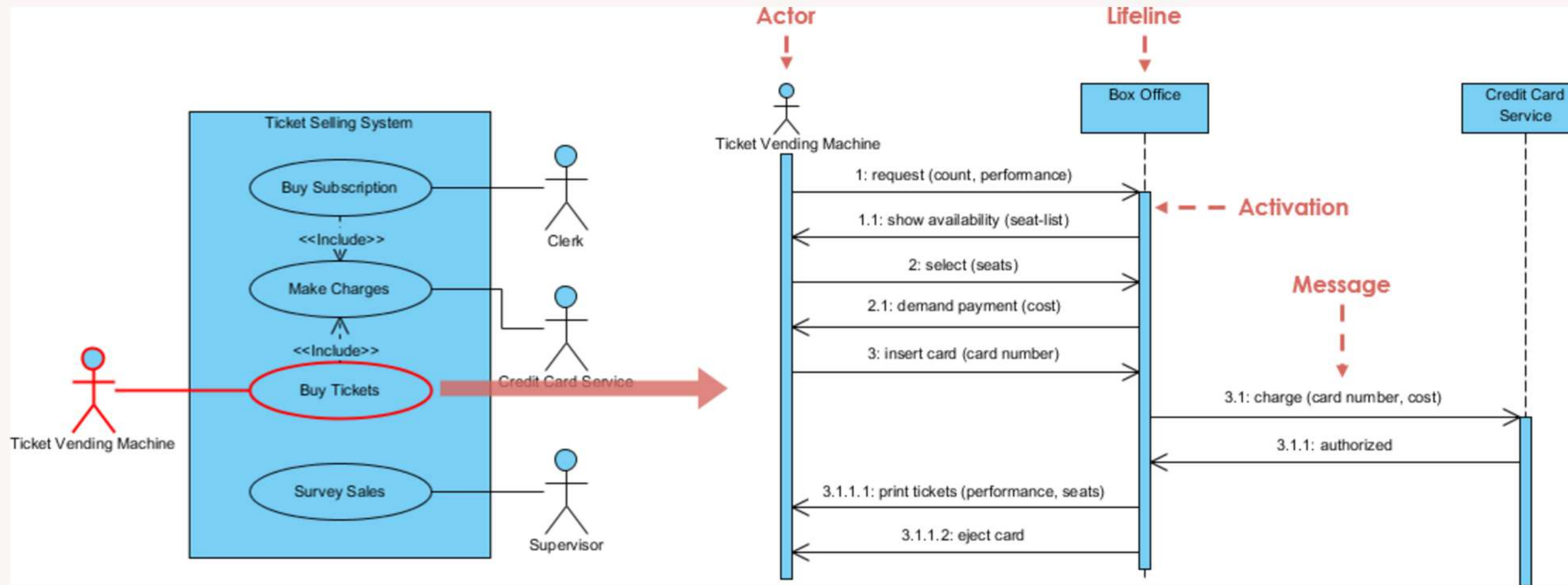
Model-driven engineering

Interaction Models



All systems involve interaction of some kind.

- This can be user interaction, which involves user inputs and outputs; User interaction modeling is important as it **helps to identify user requirements**.
- Interaction between the software being developed and other systems in its environment. Modeling system-to-system interaction highlights the **communication problems that may arise**.
- Interaction between the components of a software system. Modeling component interaction helps us understand if a **proposed system structure is likely to deliver the required system performance and dependability**.



Use Case Modelling

Captures high-level interactions between a system and external agents

Helps identify user requirements clearly and ensures the system aligns with real-world needs.

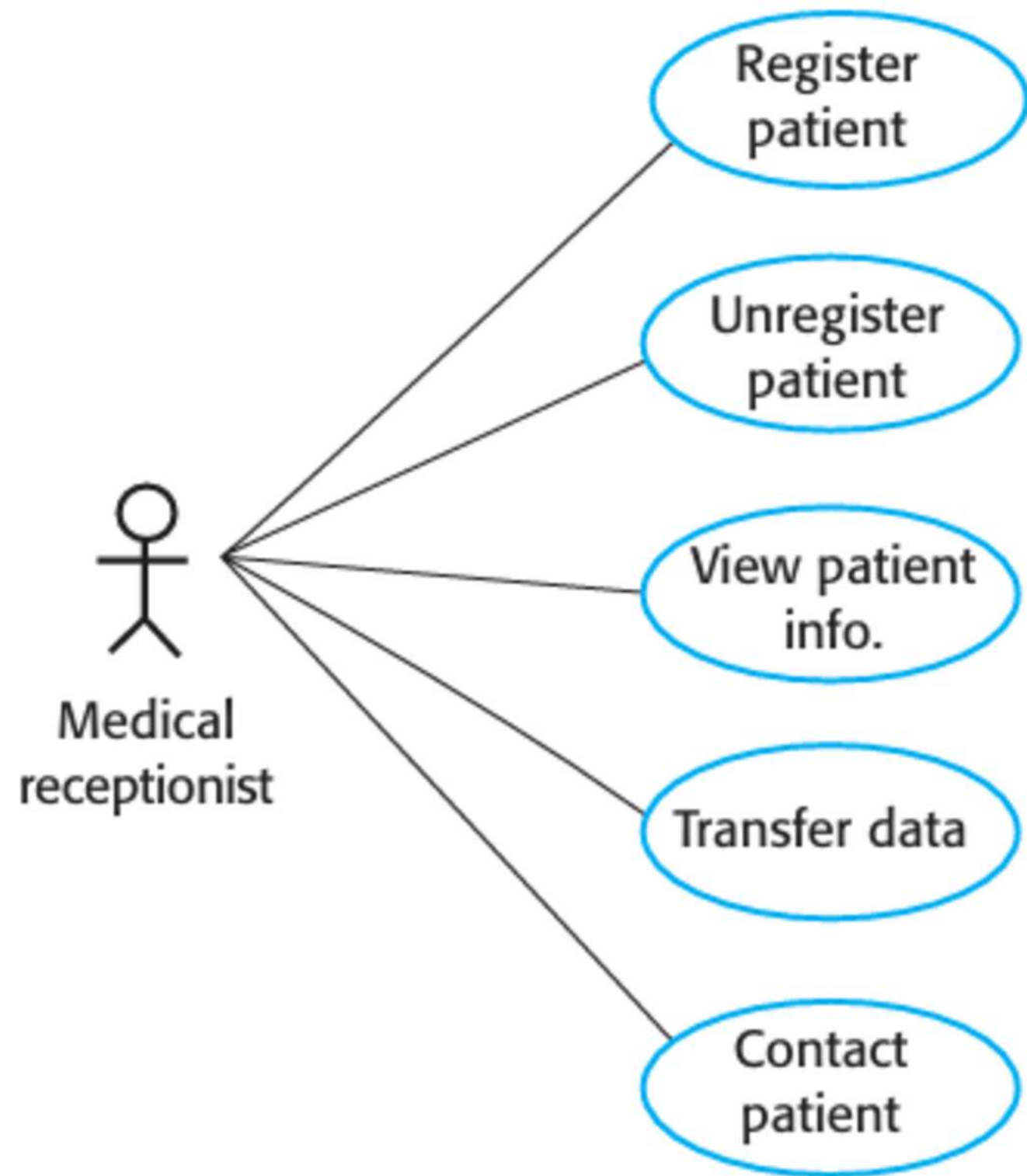
Each use case represents a discrete task that generally involves external interaction with an actor.

Sequence Diagrams

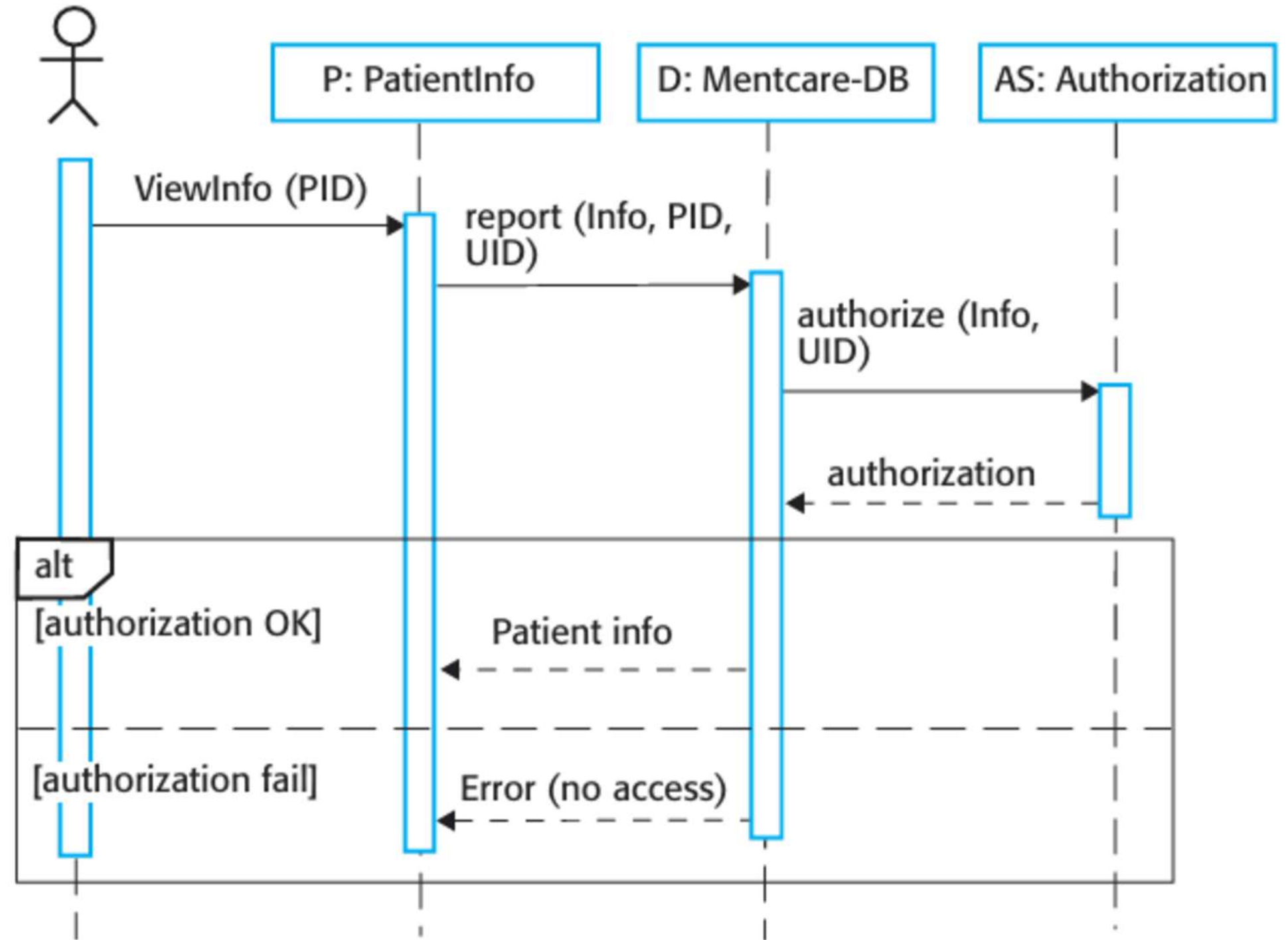
Models the flow of messages between system components over time. Fine-grained detail (step-by-step interactions).

Reveals dependencies, timing issues, and potential performance bottlenecks.

The objects and actors involved are listed along the top of the diagram, with a dotted line drawn vertically from these.



Medical Receptionist



Activity

Online Library Management System

A college library allows students to search for books and borrow them online.

System Description

- A Student logs into the system.
- The student searches for a book.
- The system displays available books.
- The student selects a book and requests to borrow it.
- The system checks availability.
- If available, the system issues the book and updates the record.
- The student can also return the book.
- A Librarian manages books (add/remove/update books).

1 Use Case Diagram

Actors:

- Student
- Librarian

Use Cases:

For Student:

- Login
- Search Book
- Borrow Book
- Return Book

For Librarian:

- Add Book
- Remove Book
- Update Book Details

System boundary: "Library Management System"

2 Sequence Diagram (Example: Borrow Book)

Objects:

- Student
- LibrarySystem
- Book
- LibraryDatabase

Flow:

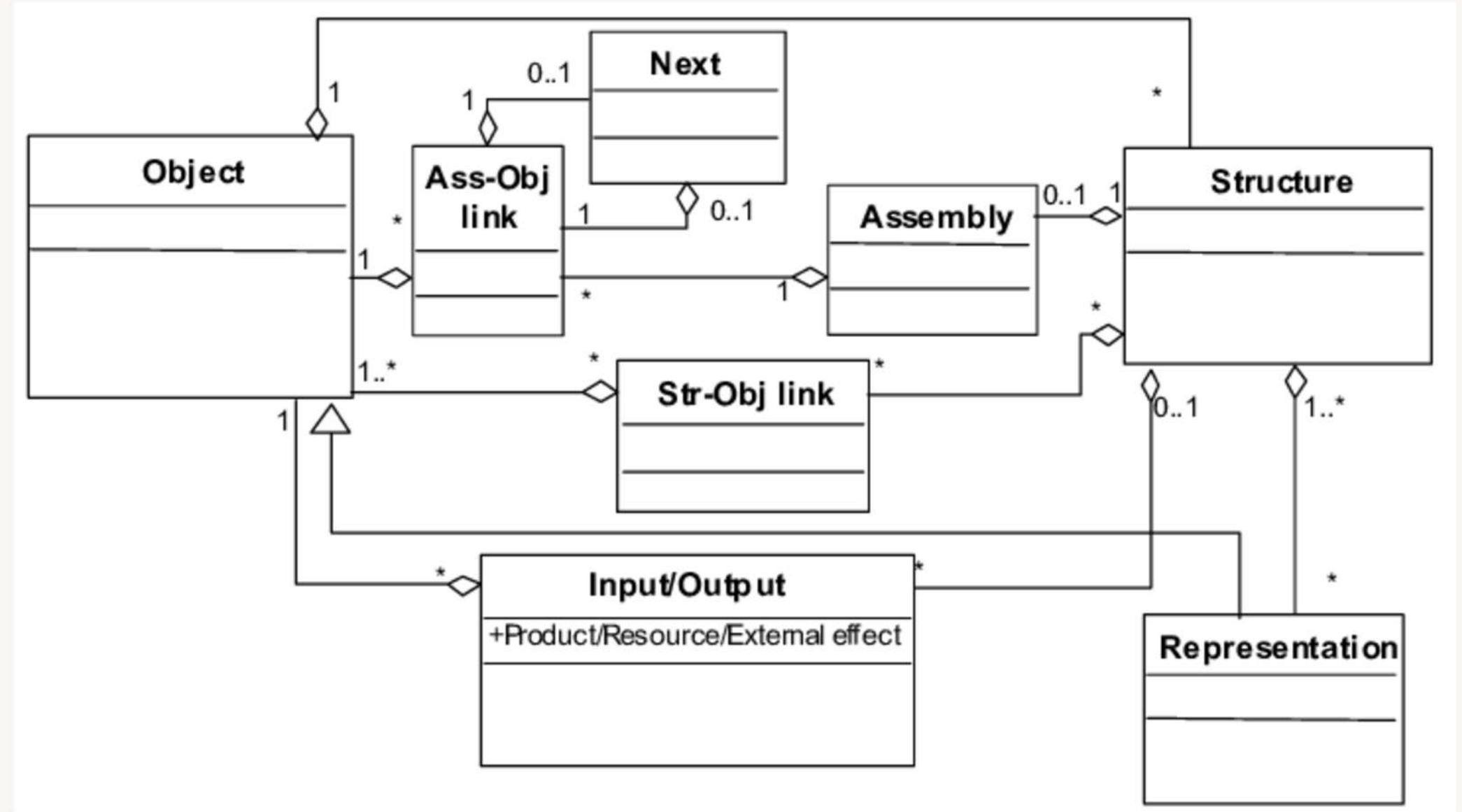
1. Student → LibrarySystem: login()
2. Student → LibrarySystem: searchBook()
3. LibrarySystem → Database: checkAvailability()
4. Database → LibrarySystem: availabilityStatus
5. Student → LibrarySystem: requestBorrow()
6. LibrarySystem → Database: updateBookStatus()
7. LibrarySystem → Student: confirmation

SYSTEM MODELLING

Context models
Interaction models
Structural models
Behavioral models
Data models
UI prototype
Model-driven engineering

Structural Models

Structural models of software - the organization of a system in terms of the elements that make up that system and of their relationships



These models help in understanding what the system consists of and how different parts are connected to each other.

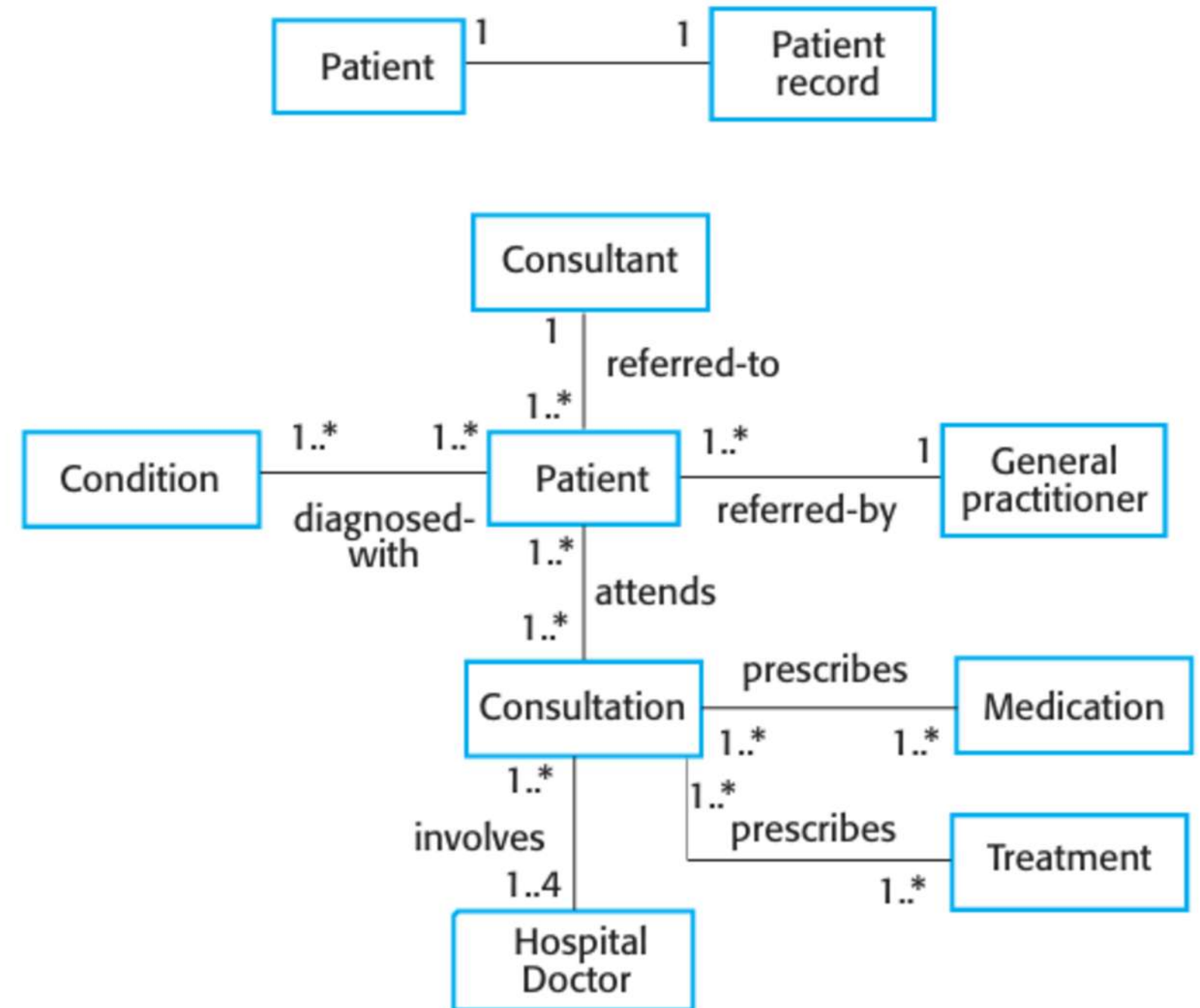
Structural models are mainly used during system architecture design and early development stages.

These models provide a clear overview of the system layout, making it easier for developers and stakeholders to understand the system's design.

Static vs Dynamic Structural Models

Static Models

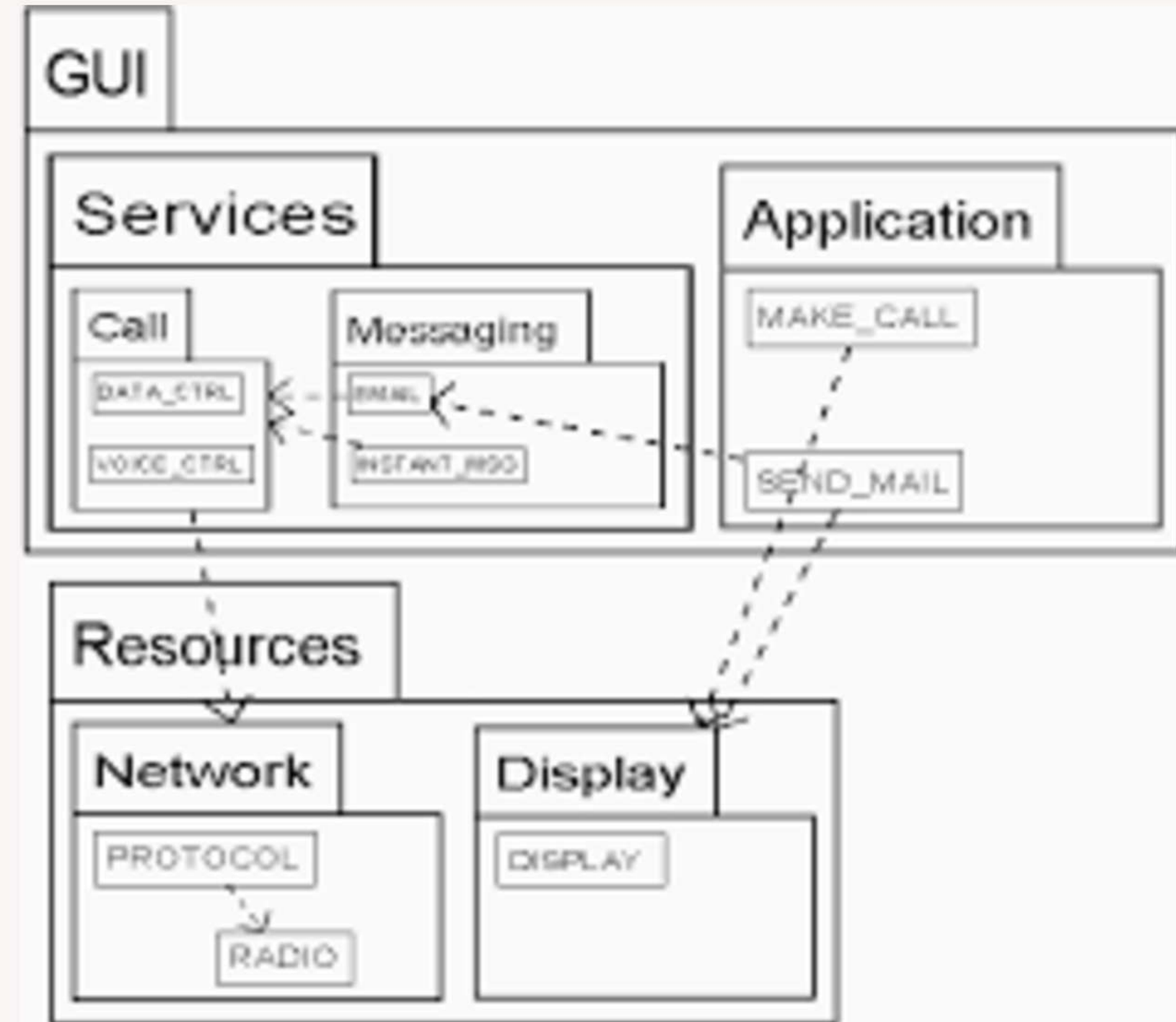
- Static models show the organization of the system design.
- They represent the system at rest, meaning they do not show how the system behaves during execution.
- These models include elements like classes, components, and their relationships.
- Examples include class diagrams and component diagrams.
- Static models are mainly used during the design phase of software development.



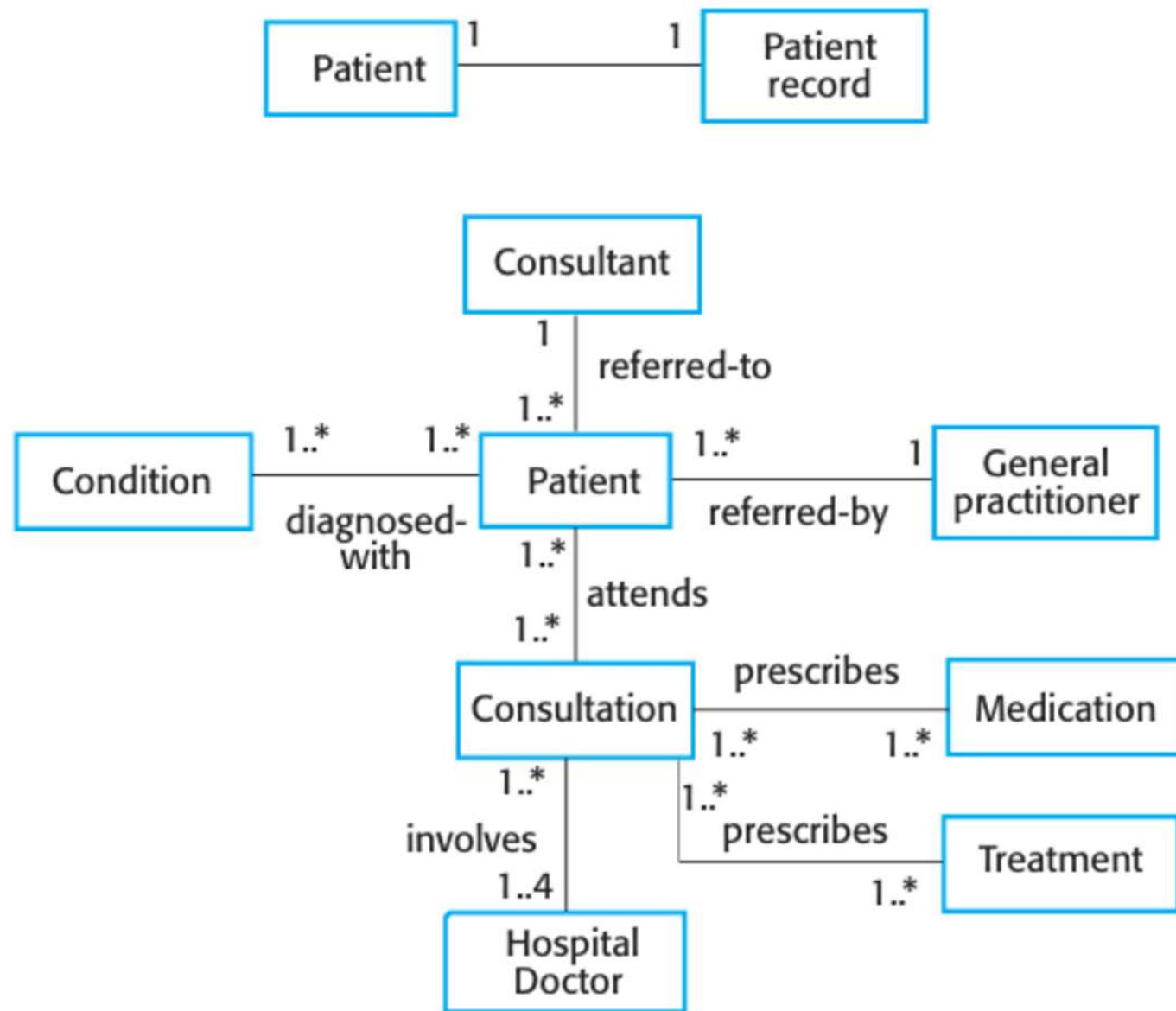
Static vs Dynamic Structural Models

Dynamic Models

- Dynamic models show how the system is organized when it is running.
- They represent interactions between objects, threads, or processes during execution.
- The dynamic structure of a system may differ significantly from its static structure.
- For example, a system may have five classes statically, but during execution, dozens of objects may interact dynamically.



Class Diagram



- Class diagrams are used in object-oriented systems to represent the static structure of a system.
- They show the classes in a system and the associations between these classes.
- A class represents a blueprint or general definition of an object type.
- An association is a link between two classes indicating a relationship.
- Class diagrams are widely used during requirements analysis and early system design.

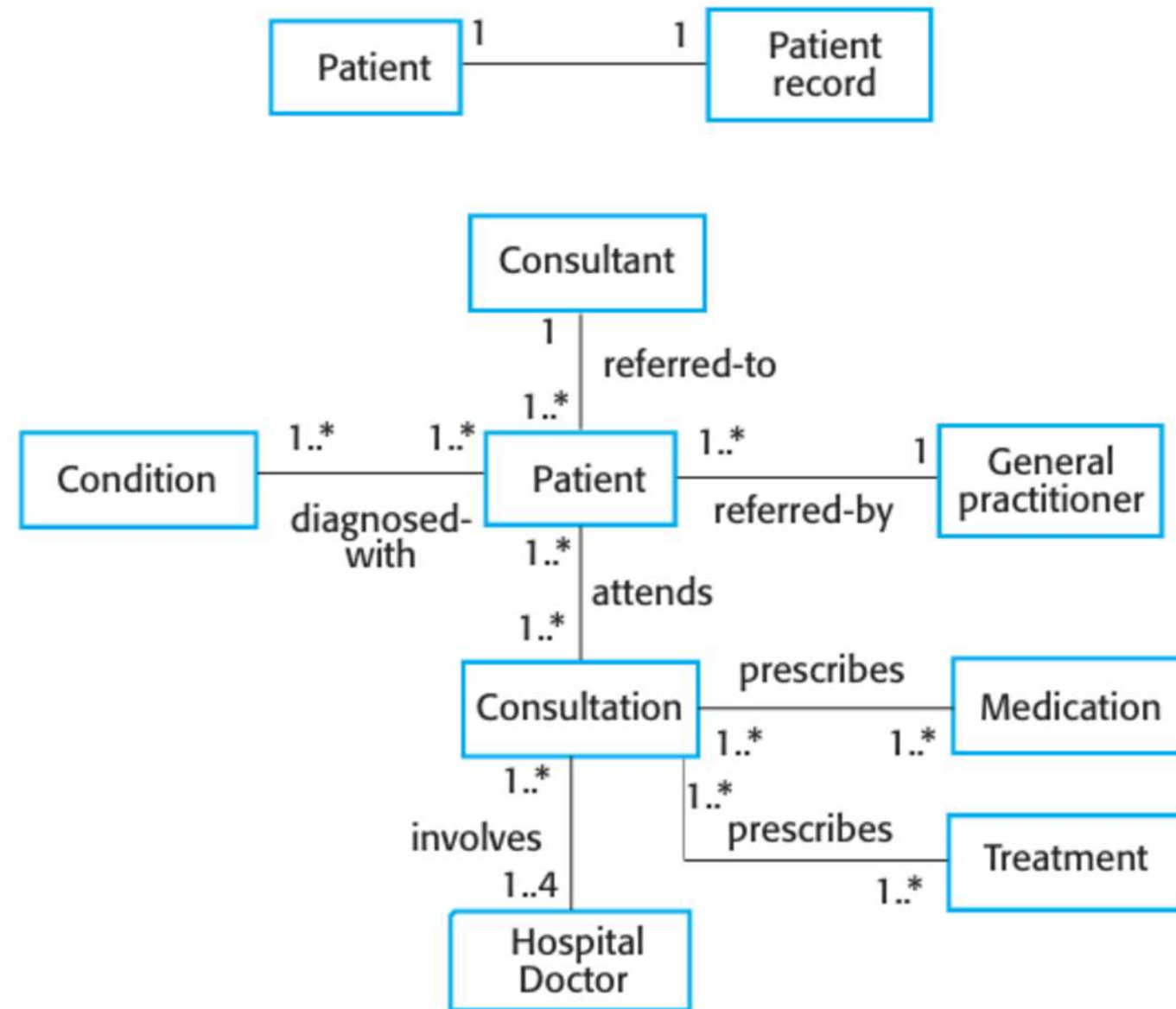
Class Diagram

Object Classes

Represent a general definition of system objects; in early stages they model real-world entities (e.g., Patient, Doctor), while later they may represent system-level components (e.g., database handlers). They organize both data and behavior in object-oriented design.

Associations

Define relationships between classes and are shown as lines in a class diagram. They indicate how objects of one class are connected to objects of another, and associations may be named to describe the relationship.

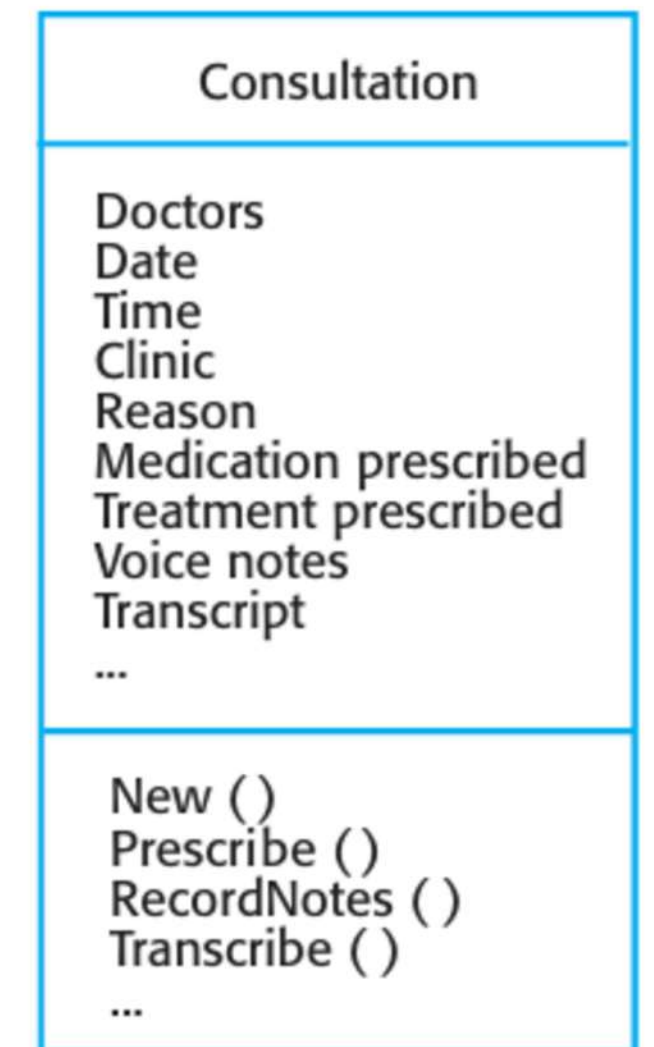


Multiplicity (Cardinality)

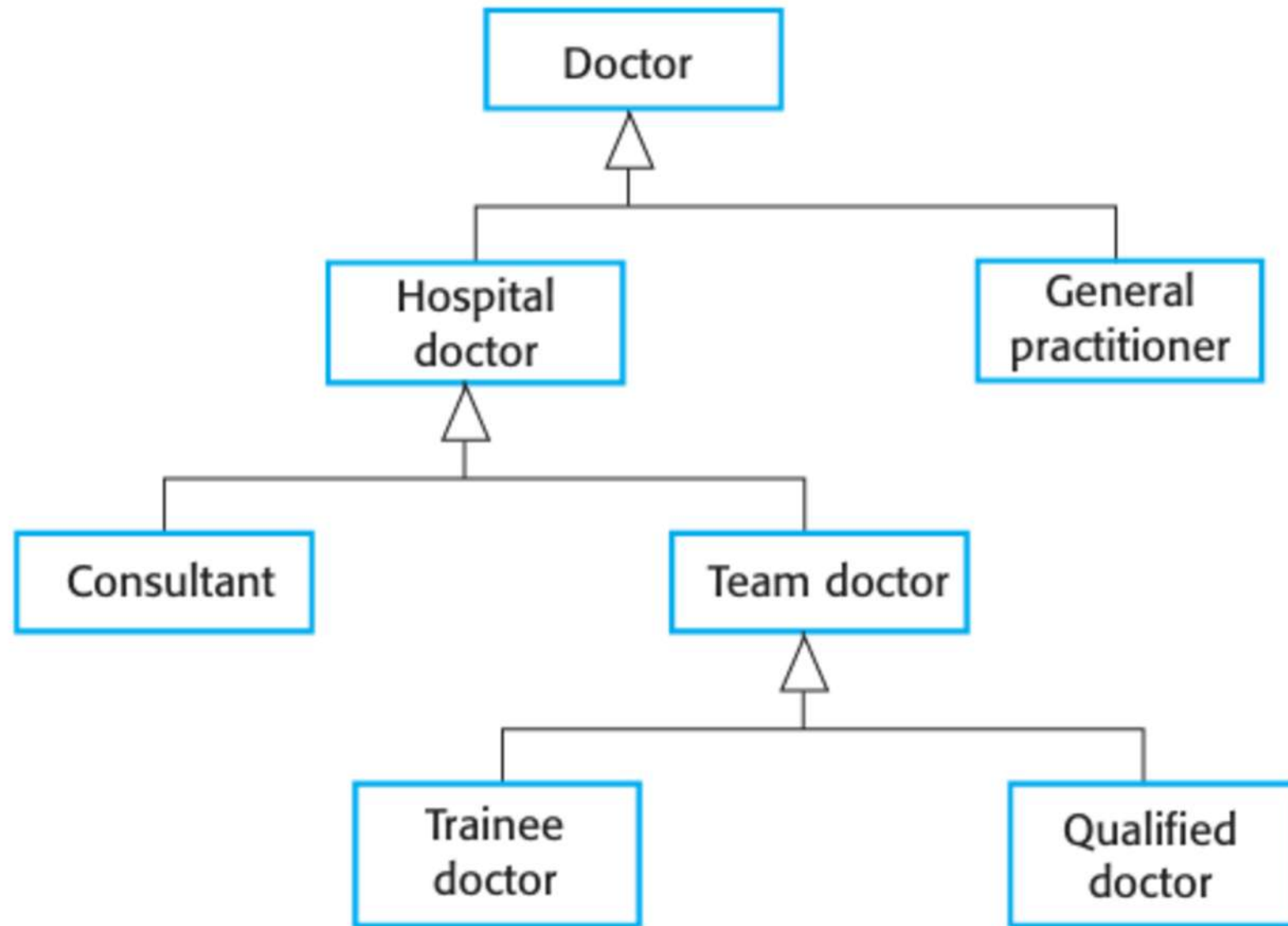
Specifies how many objects participate in a relationship (e.g., 1:1, 1..*, *). It is shown at the ends of association lines and helps define system constraints clearly.

Structure of a UML Class

A class is represented by a three-part rectangle containing the class name (top), attributes (middle), and operations/methods (bottom), making the model closer to real implementation.



Class Diagram- Generalization

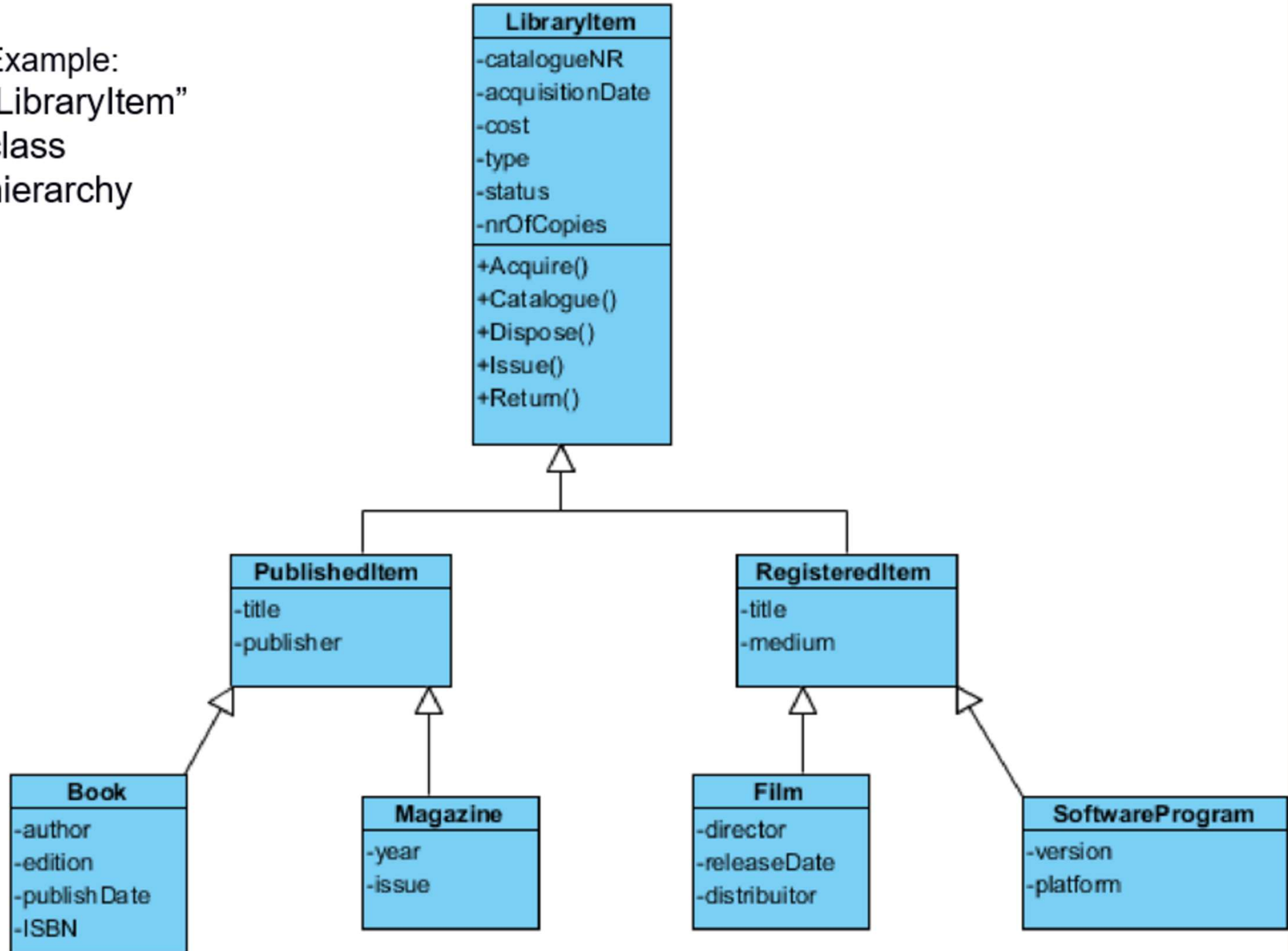


Generalization Technique to manage complexity by abstracting common characteristics of more classes.

Generalization is similar to the “is-a” relationship in semantic data models.

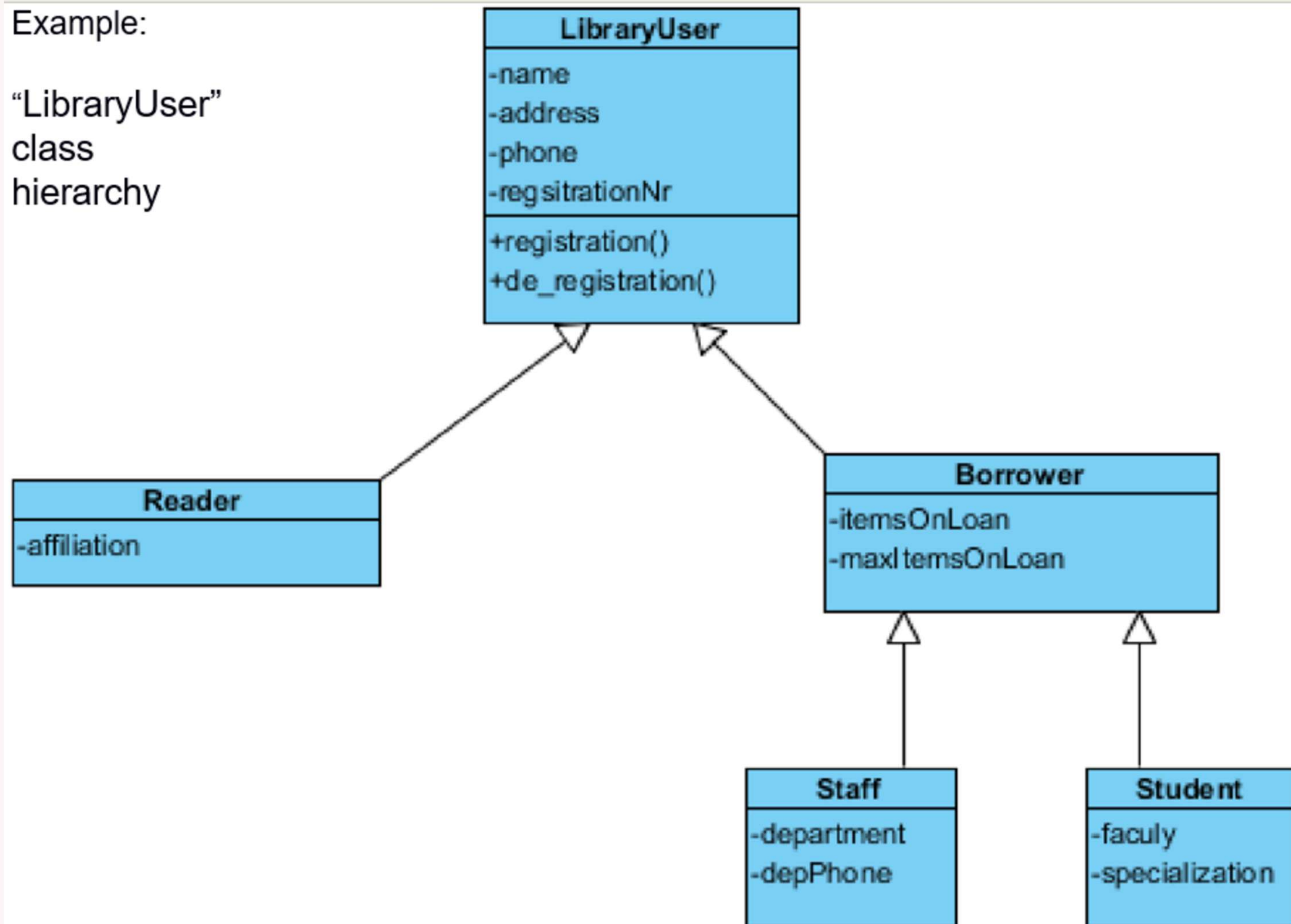
- In object-oriented languages (ex. Java) generalization is implemented using the class inheritance mechanisms built into the language.
- In a generalization, the attributes and operations associated with the superclass are also associated with all its subclasses.
- A subclass inherits all the attributes and operations from its superclasses. Subclasses can add more specific attributes and operations.

Example:
"LibraryItem"
class
hierarchy

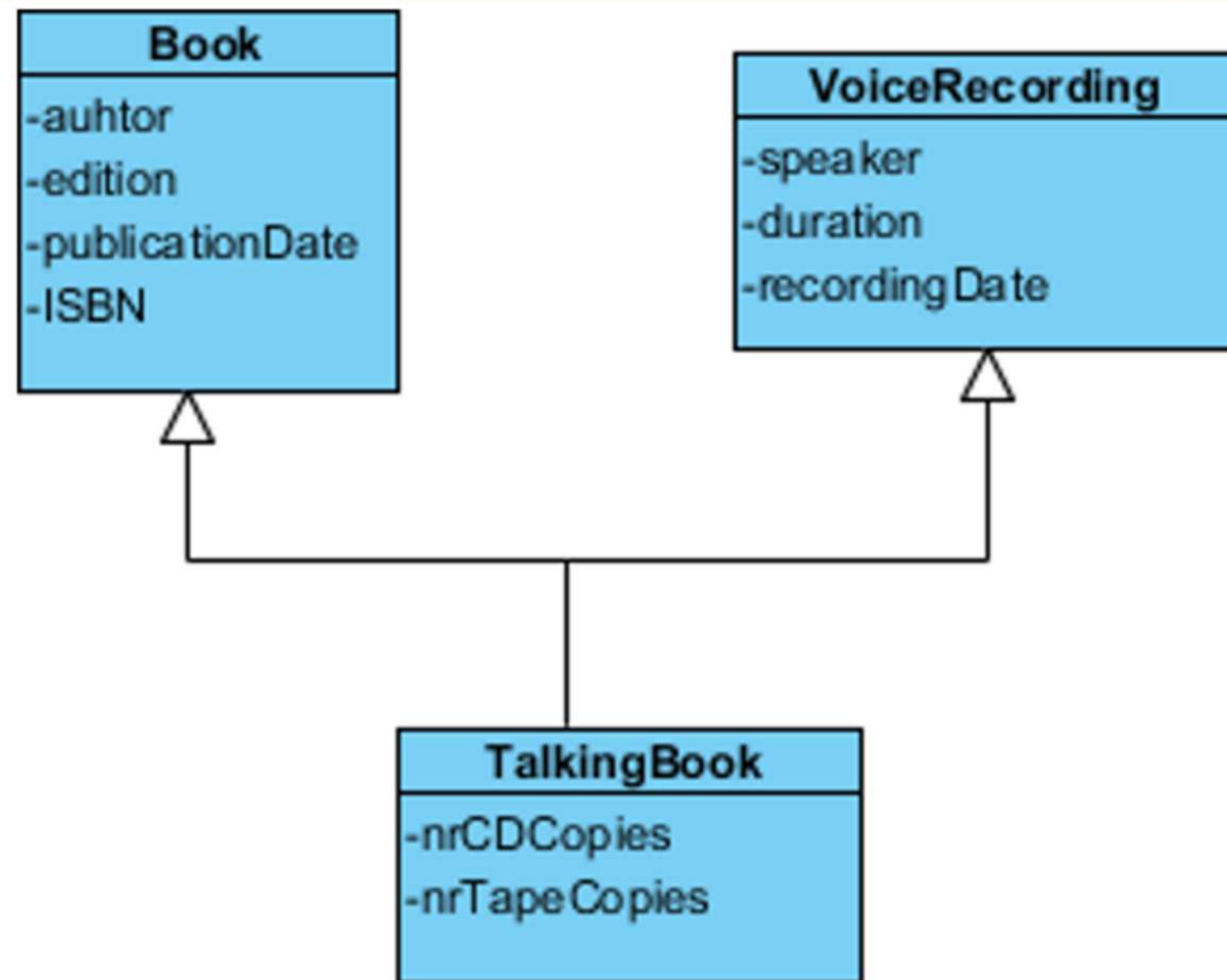


Example:

“LibraryUser”
class
hierarchy



Class Diagram- Multiple Inheritance

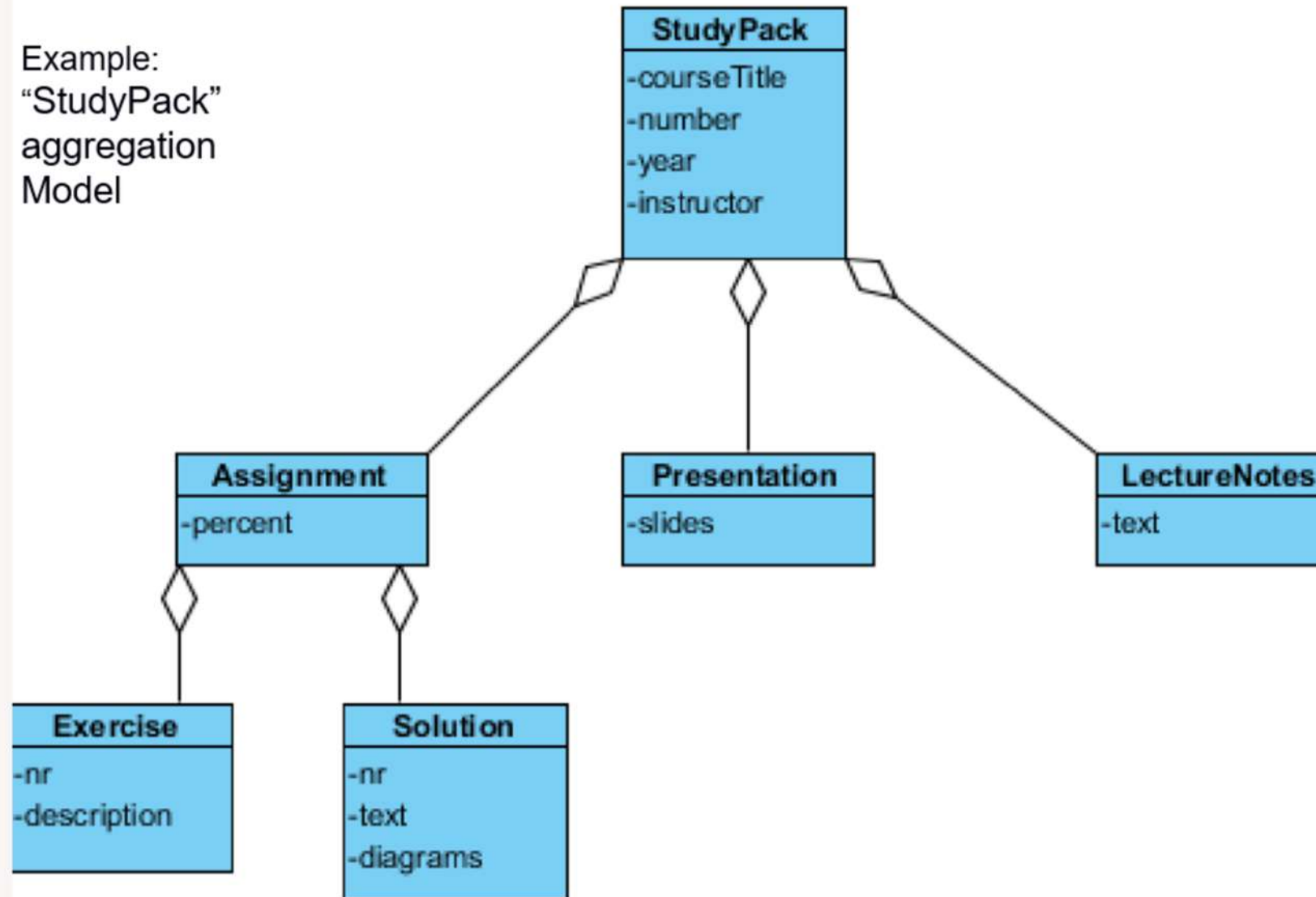


Multiple Inheritance

Rather than inheriting the attributes and services from a single parent class, a system which supports multiple inheritance allows object classes to inherit from several super-classes

- This can lead to semantic conflicts where attributes/services with the same name in different super-classes have different semantics.
- Multiple inheritance makes class hierarchy re-organization more complex

Example:
“StudyPack”
aggregation
Model



Class Diagram- Aggregation

Aggregation shows how classes that are collections are composed of other classes.

Aggregation models are similar to the “part-of” or “Has-a” relationship in semantic data models.

The object which represents the collection keeps a data structure of type Collection containing the references to the objects that form the aggregate.

The element in the collection may keep a reference to the object representing the collection

Key Differences

Aspect	Multiple Inheritance	Aggregation
Relationship type	Is-a	Has-a
Purpose	Share behavior/attributes across hierarchies	Represent whole–part relationships
Independence	Child depends on parents	Parts can exist independently of the whole
Example	<code>FlyingCar</code> inherits <code>Car</code> + <code>Airplane</code>	<code>Library</code> has <code>Books</code>

SYSTEM MODELLING

Context models

Interaction models

Structural models

Behavioral models

Data models

UI prototype

Model-driven engineering

Behavioral Models

They describe the dynamic behavior of a system as it executes.

Purpose: *To show what happens when the system responds to a stimulus (something coming from its environment).*

Data-driven modelling

Models show the sequence of actions involved in processing input data and generating an associated output

A phone billing system receives call records, calculates charges, and generates bills.

Event-driven modelling

Models show how the system responds to events

A telephone switching system responds to events like “handset lifted” by generating a dial tone, or “keys pressed” by capturing the number.



Behavioral Models

Data-driven Modelling

Models show the sequence of actions involved in processing input data and generating an associated output

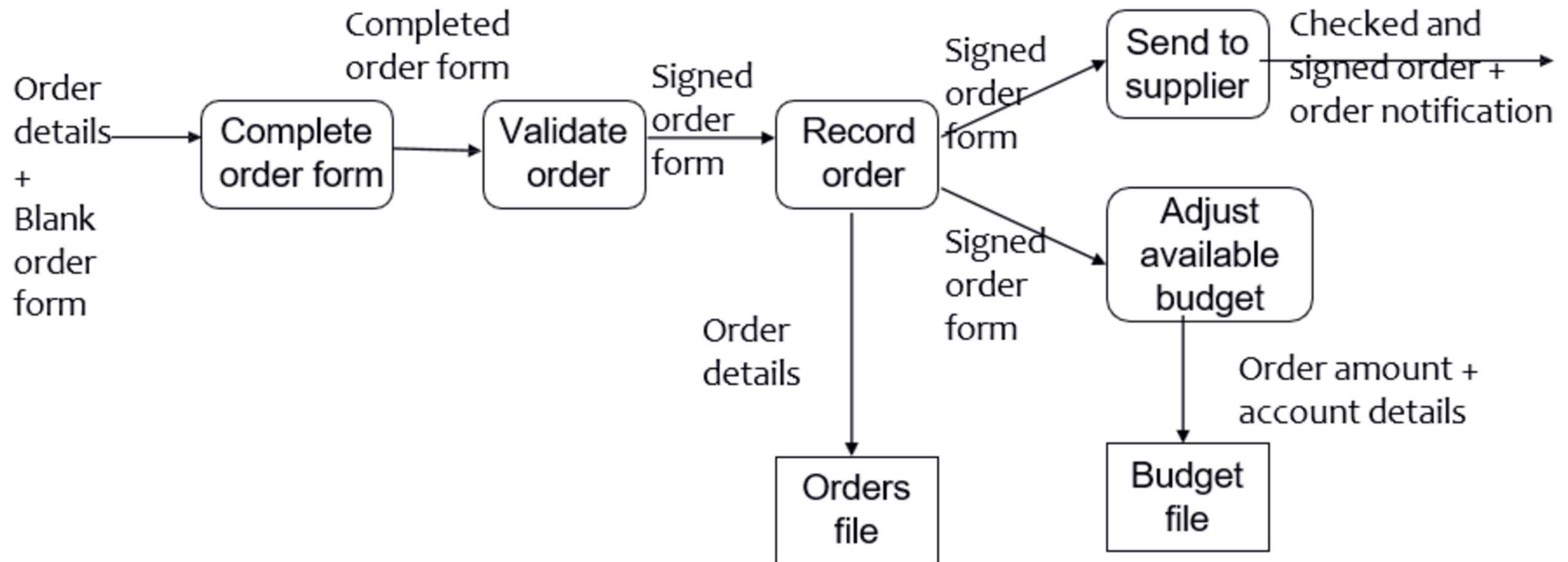
Show how data is processed as it moves through the system

Types of diagrams:

- Data flow diagram (DFDs)
- UML activity diagram

- *Tracking and documenting how the data is associated with a process is helpful to develop an overall understanding of the system.*
- *Data flow diagrams may also be used in showing the data exchange between a system and other systems in its context.*

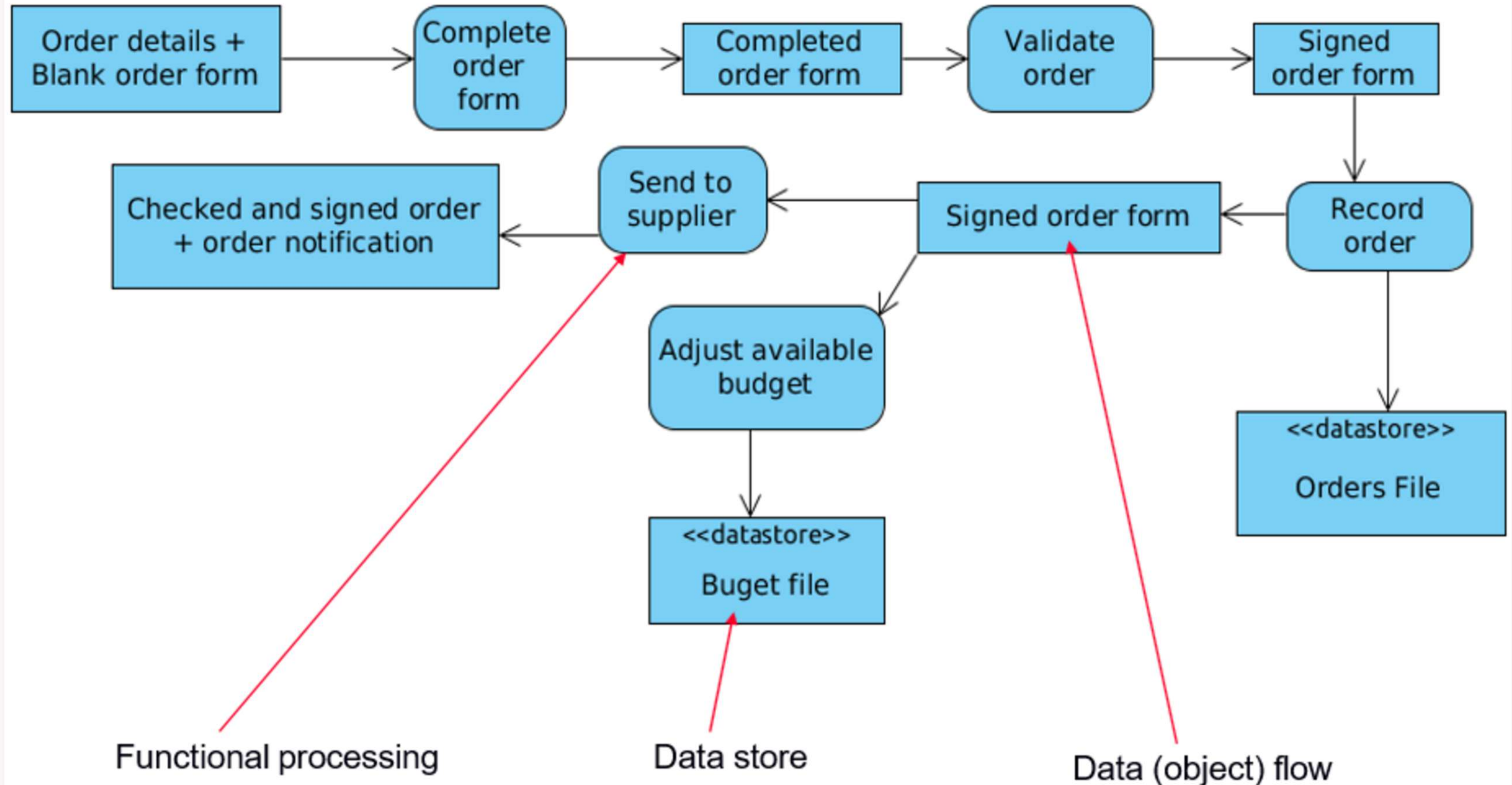
Example DFD: Order processing DFD



Notation:

- Round rectangle – functional processing
- Rectangle – data store
- Labelled arrow – data flow.

Example Activity Diagram: Order processing activity diagram



Behavioral Models

Event-driven Modelling

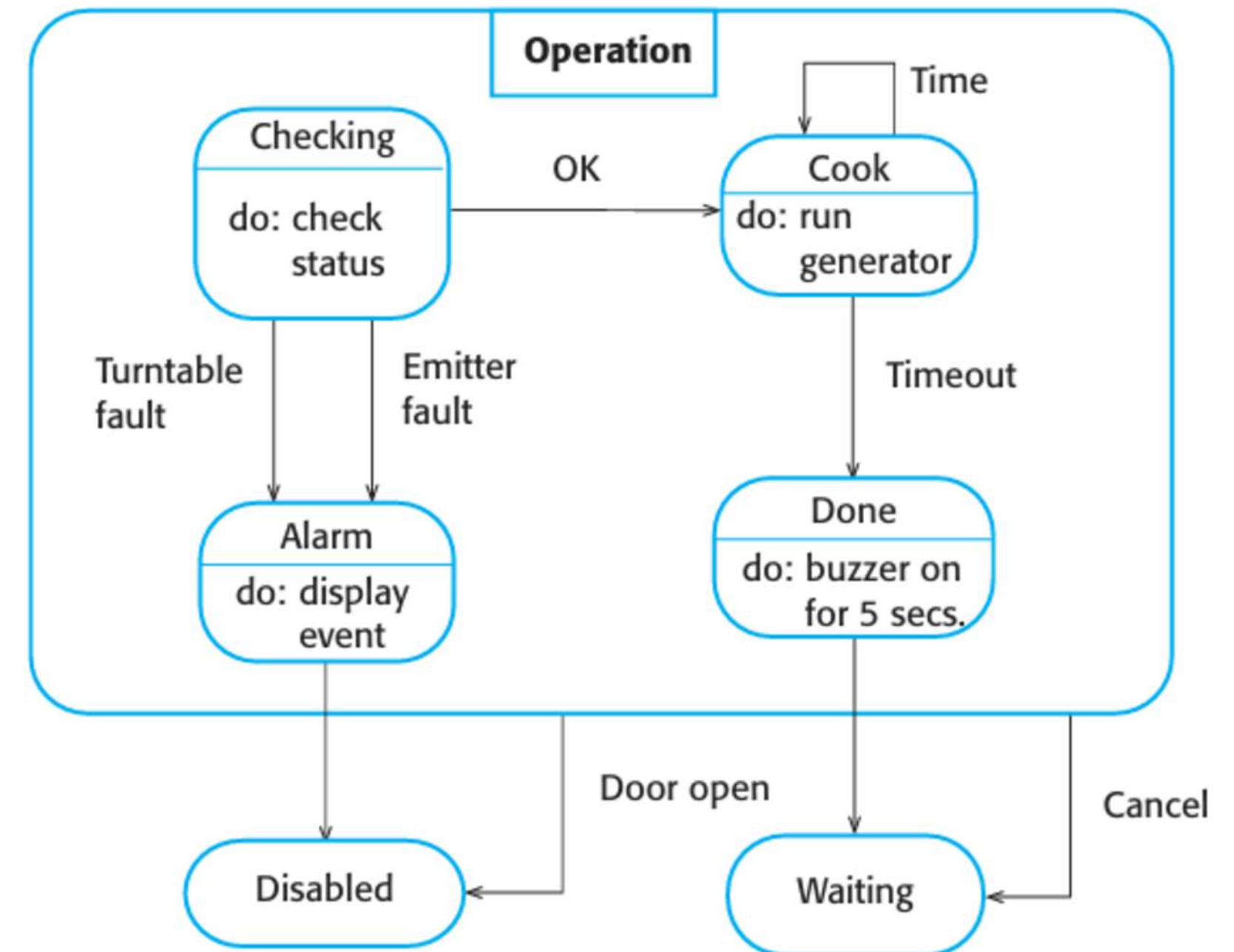
Models show how the system responds to events

Modelling elements:

- *System states as nodes*
- *Events as edges between these nodes.*
- *When an event occurs, the system moves from one state to another = makes a state transition.*

Model the behavior of the system in response to external and internal events.

Shows the system's responses to stimuli (so are often used for modelling real-time systems).



Behavioral Models

State machine diagram (SMD)

Describes a behavior or a behavioral feature.

Modelling elements:

- *States*
- *Transitions connecting states*
- *Triggers for the transitions*
- *Activities performed in the execution of transitions*
- *Activities performed throughout the duration of each state*

Behavioral Models

State machine diagram (SMD)

Describes a behavior or a behavioral feature.

What is a State?

- A state describes the condition of an object at a particular time.
- It is expressed in terms of:
 - Attributes → the values of the object's properties (e.g., a microwave's timer = 30 seconds, door = closed).
 - Behavior → what the object is currently doing (e.g., cooking, waiting, checking).

What is object State?

- The object state is the cumulative result of its behavior.
- It is defined by the values of its attributes and relationships.
- Example: A bank account object's state could be: balance = ₹10,000, status = active, owner = Shishir.
- These attribute values together define the account's current state.

Inside a state an object may:

- execute an activity
- wait for an event
- fulfill a condition.

Object: Microwave Oven

```
+-----+
|   Checking   |
| do: check status |
| Attributes:   |
| - timeRemaining = 0 |
| - powerLevel = 0   |
+-----+
      |
      | [OK] v
+-----+
|   Cook       |
| do: run generator |
| Attributes:   |
| - timeRemaining > 0 |
| - powerLevel = High |
+-----+
```

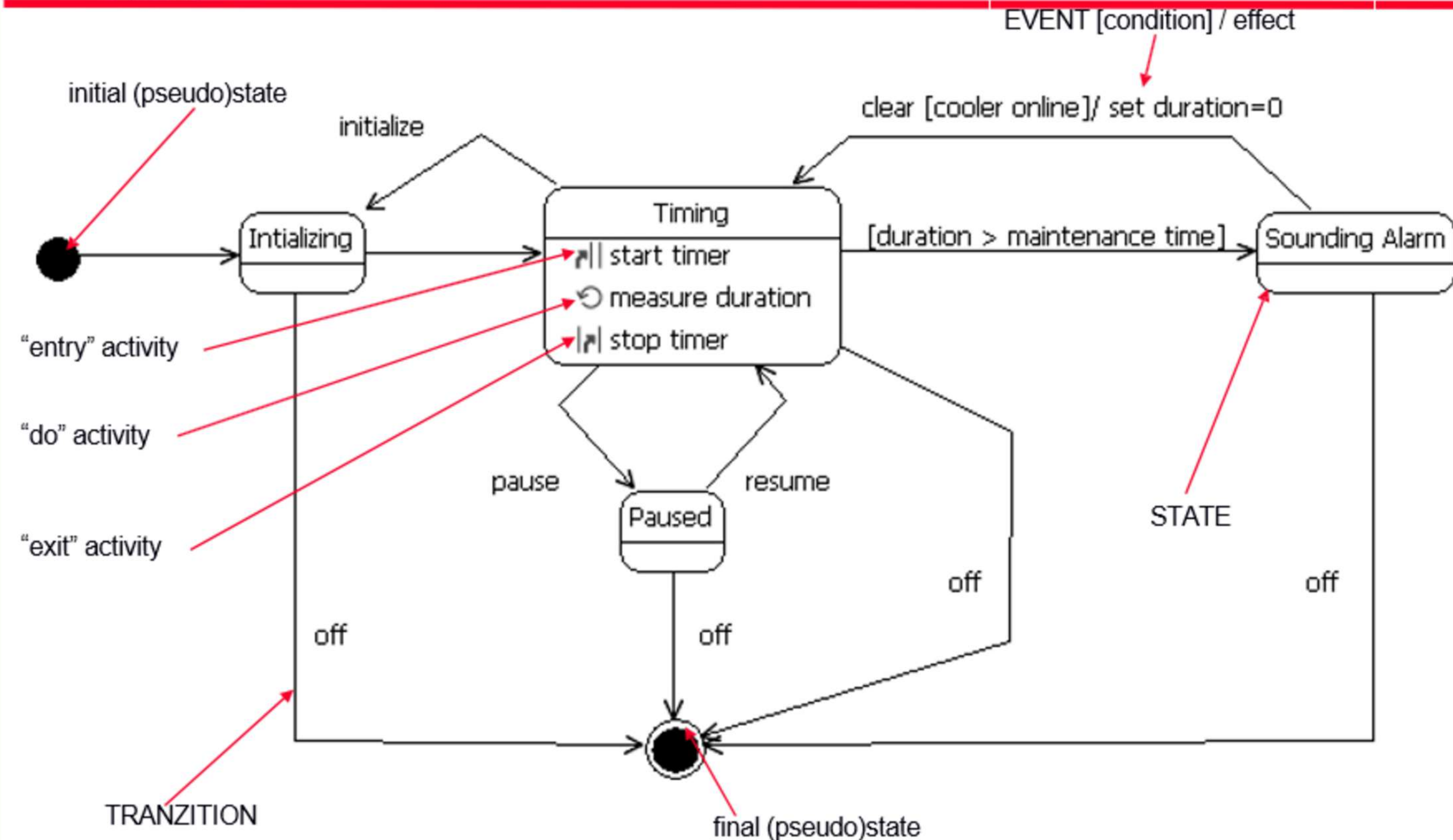

MAIN MODELLING ELEMENTS:

- state
- transition
- event

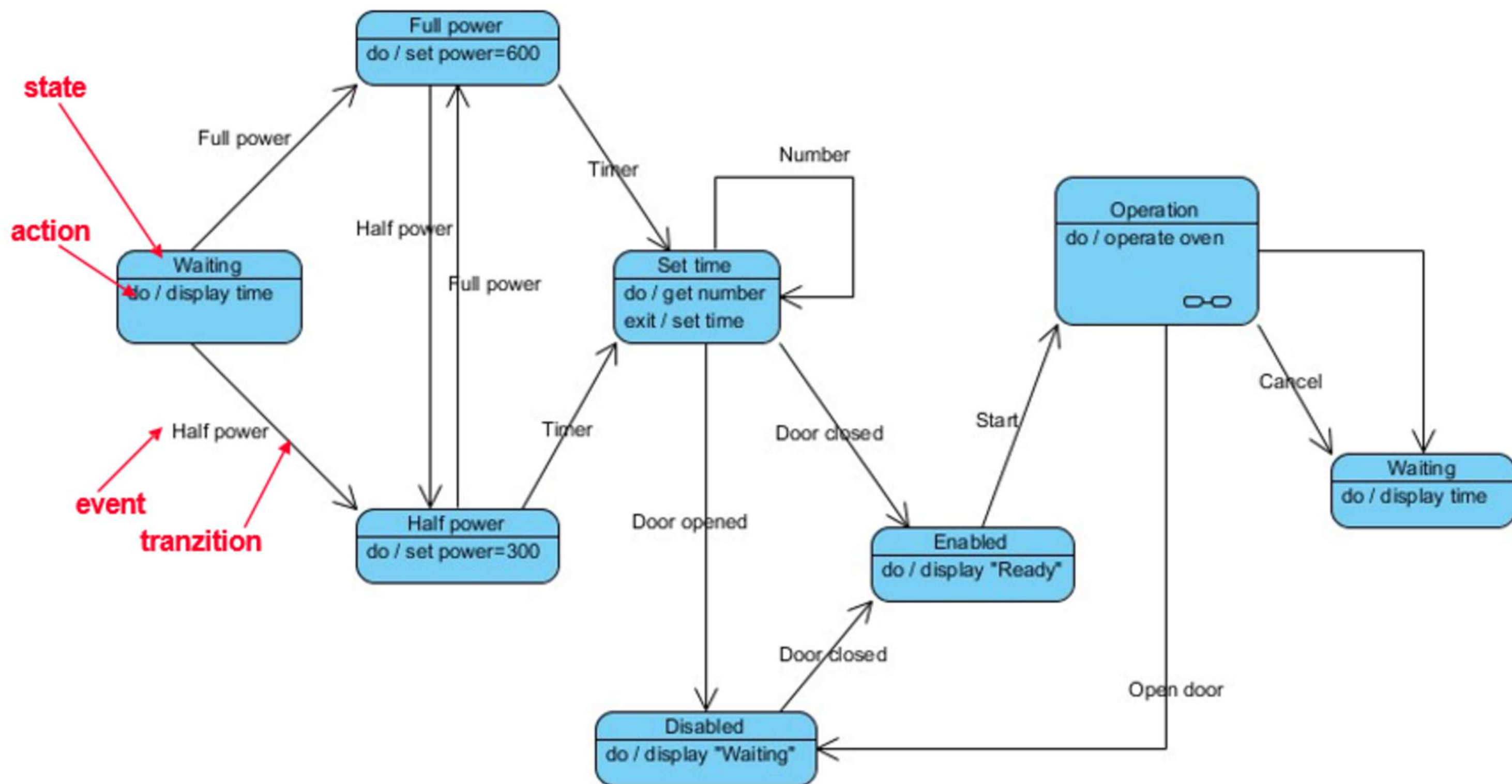
Data-driven modelling

Event-driven modelling

State machine diagram (SMD)



Example: Microwave oven model



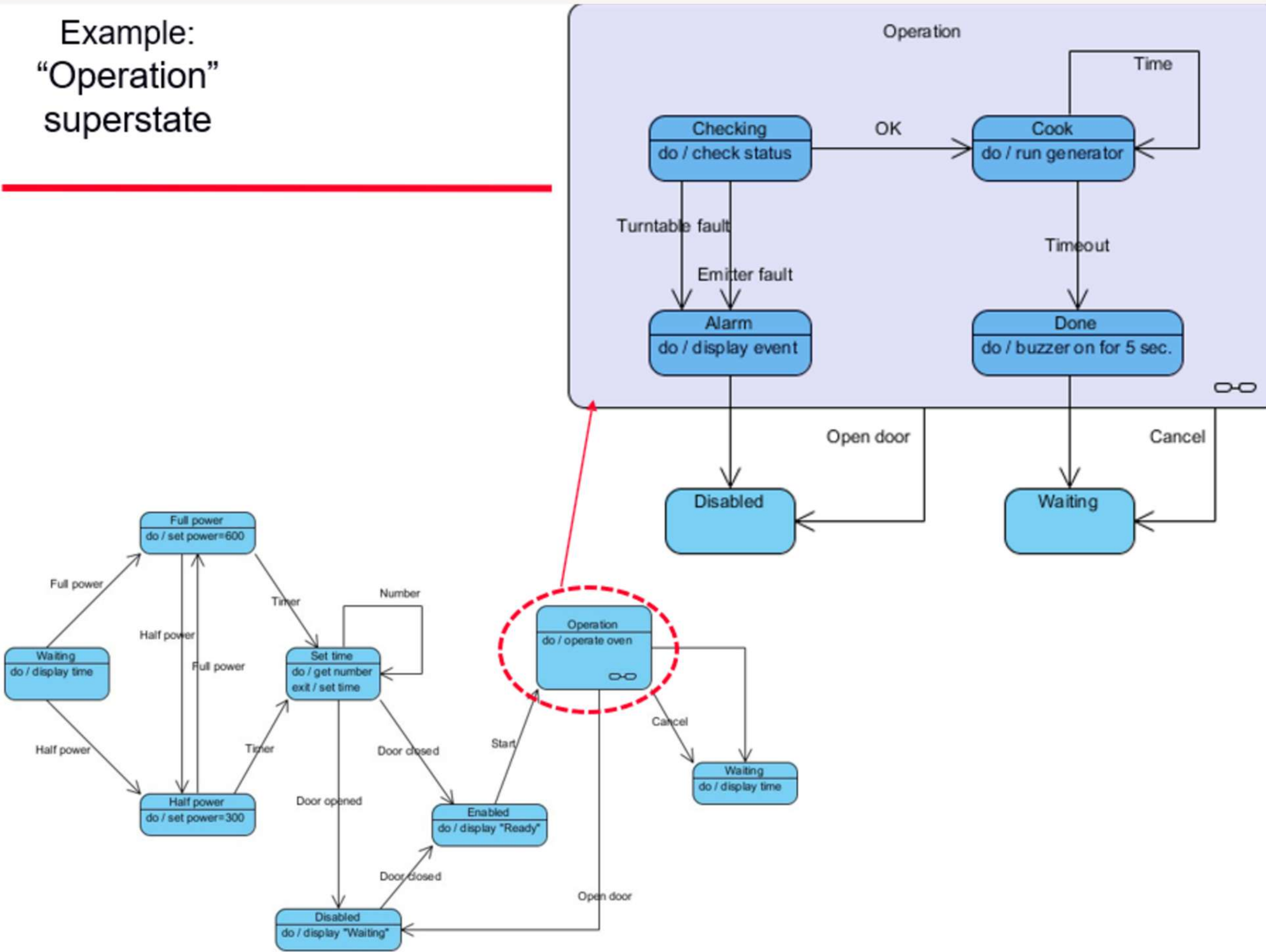
Example: Microwave oven state description

State	Description
Waiting	The oven is waiting for input. The display shows the current time.
Half power	The oven power is set to 300 watts. The display shows 'Half power'.
Full power	The oven power is set to 600 watts. The display shows 'Full power'.
Set time	The cooking time is set to the user's input value. The display shows the cooking time selected and is updated as the time is set.
Disabled	Oven operation is disabled for safety. Interior oven light is on. Display shows 'Not ready'.
Enabled	Oven operation is enabled. Interior oven light is off. Display shows 'Ready to cook'.
Operation	Oven in operation. Interior oven light is on. Display shows the timer countdown. On completion of cooking, the buzzer is sounded for 5 seconds. Oven light is on. Display shows 'Cooking complete' while buzzer is sounding.

Example: Microwave oven stimuli

Stimulus	Description
Half power	The user has pressed the half power button
Full power	The user has pressed the full power button
Timer	The user has pressed one of the timer buttons
Number	The user has pressed a numeric key
Door open	The oven door switch is not closed
Door closed	The oven door switch is closed
Start	The user has pressed the start button
Cancel	The user has pressed the cancel button

Example: “Operation” superstate



Superstate

- Notion that encapsulates a number of separate states.
- Looks like a single state on a high-level model.
- Is expanded in more detail in a separate diagram

NO MORE

IN NEXT CLASS!